Tema 4 – Transformación del Modelo E/R al Modelo Relacional

IES Ntra. Sra. de los Remedios (Ubrique) · 1º DAW – Bases de Datos
J. Carlos Moreno · Curso 2026/2027

Índice

- 4.1 Introducción
- 4.2 Transformación de entidades
  - 4.2.1 Atributos de las entidades
  - 4.2.2 Atributos compuestos
  - 4.2.3 Atributos multivaluados
  - 4.2.4 Caso práctico I
- 4.3 Relaciones M:N
- 4.4 Relaciones 1:N
- 4.5 Relaciones 1:1
- 4.6 Relaciones ternarias
- 4.7 Relaciones n-arias
- 4.8 Relaciones reflexivas
- 4.9 Relaciones débiles
- 4.10 Jerarquías
- 4.11 Resumen de las reglas de transformación
- 4.12 Ampliaciones
- 4.13 Referencias normativas

4.1 Introducción

En el tema anterior estudiamos un modelo conceptual que permite describir la información que se desea almacenar en una base de datos: el modelo Entidad-Relación (E/R). Su ventaja es que es independiente del modelo lógico sobre el que se implantará finalmente la base de datos. Cuando ese modelo lógico es el modelo relacional, el paso del diagrama E/R al esquema relacional se realiza mediante unas reglas sencillas, llamadas **reglas de transformación del modelo E/R al modelo relacional**.

Antes de comenzar conviene recordar las diferencias entre ambos modelos:

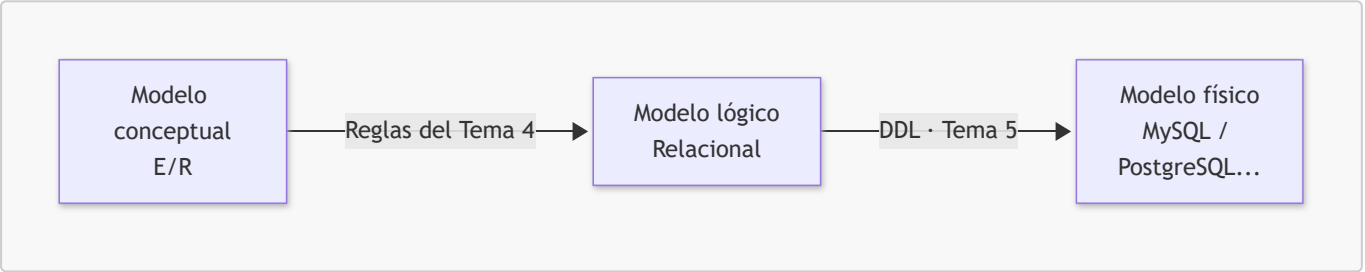
- El **modelo E/R** trabaja a nivel **conceptual**: establece cuáles son las entidades (fuertes y débiles) que intervienen y las relaciones existentes entre ellas, pero no dice nada sobre cómo se almacenan los datos.
- El **modelo relacional** trabaja a nivel **lógico**: representa la información como la percibe el usuario y la gestiona el SGBD, empleando casi exclusivamente el concepto de **tabla**. Se trata, por tanto, de pasar de describir el mundo mediante entidades y relaciones a describirlo mediante tablas.

El diseño lógico se puede dividir en dos etapas:

- **Diseño lógico estándar.** Partiendo del esquema conceptual (en nuestro caso el modelo E/R) se elabora un esquema lógico basado en el modelo de datos que soporte el SGBD elegido (generalmente, el modelo relacional). Es el objetivo de este tema.
- **Diseño lógico específico.** Una vez obtenido el diseño lógico estándar, se adapta al lenguaje de definición de datos (DDL) del SGBD elegido (MySQL, PostgreSQL, Oracle, SQL Server, etc.). Se trabajará en el Tema 5.

Resultados de aprendizaje. Este tema trabaja principalmente el **RA6** («Diseña modelos relacionales normalizados interpretando diagramas entidad/relación»), en particular los criterios b) a f) (identificar tablas, campos, relaciones y campos clave, y aplicar reglas de integridad) y h) (documentar las restricciones que no pueden plasmarse en el diseño lógico). Sienta además las bases del RA2 (creación de tablas y restricciones en SQL, Tema 5) y prepara el criterio g) (normalización, tema siguiente).

> Ilustración 4-1. El diseño pasa del modelo conceptual (E/R), al lógico (modelo relacional) y de este al físico (DDL del SGBD).



4.2 Transformación de entidades

Cada **tipo de entidad** se convierte en una **relación o tabla** en el modelo relacional. La tabla se llamará igual que el tipo de entidad de donde proviene.

4.2.1 Transformación de los atributos de las entidades

Por regla general, cada atributo de una entidad se transforma en un atributo (columna) de la tabla a la que ha dado lugar la entidad, aunque la transformación depende del tipo de atributo:

- **Identificador principal o atributos clave.** El (o los) atributo(s) clave pasan a ser la **clave primaria** de la tabla. Se representa en negrita y subrayado, y se implementa con `PRIMARY KEY`.
- **Atributos obligatorios.** No tienen representación especial, solo su nombre. Se implementan con `NOT NULL`.
- **Atributos opcionales.** Se muestra el nombre seguido de un asterisco (*). Admiten `NULL`.
- **Claves alternativas.** Se representan con doble subrayado y se implementan con `UNIQUE` (y `NOT NULL` si son obligatorias).
- **Atributos derivados.** Se marcan con el sufijo `-d`. Su valor se calcula a partir de otros datos, por lo que no es necesario almacenarlo; si se implementan, se obtienen mediante una vista, una columna generada o un procedimiento, función o trigger.
- **Atributos compuestos y multivaluados.** Requieren un tratamiento especial (apartados siguientes).

> Ilustración 4-2. Entidad Artículo con identificador `id`, atributos obligatorios (Nombre, CodigoBarras, FechaAlta), opcional (Stock), alternativo (CodigoBarras) y derivado (TiempoAlmacen).

A partir de la figura 4-2, la transformación de la entidad Artículo quedaría:

> Articulo(**#id**, Nombre, CodigoBarras, FechaAlta, Stock*, TiempoAlmacen-d)

Cuando se implemente en SQL, la tabla tendrá las siguientes restricciones:

- `PRIMARY KEY` en `id`.
- `UNIQUE` en `CodigoBarras`.
- `NOT NULL` en `FechaAlta`, `CodigoBarras` y `Nombre` (`Stock` es opcional y admite `NULL`).
- `TiempoAlmacen` es un atributo derivado: no se almacena, o se mantiene mediante un trigger o una columna generada.

Convenio de notación utilizado en este tema

Elemento	Notación	Ejemplo
Clave primaria	**Negrita y subrayado**, precedida de `#`	**#id**
Clave alternativa	Doble subrayado	CodigoBarras
Clave ajena	Subrayado **punteado** (sin negrita)	id_pais
Clave ajena que forma parte de la clave primaria	Negrita, subrayado y `#` (ver nota)	**#id_articulo**
Atributo opcional	Asterisco	Stock*
Atributo derivado	Sufijo `-d`	TiempoAlmacen-d

> **Alternativa compatible con Markdown.** El subrayado punteado se obtiene con HTML/CSS (`<span style="text-decoration: underline dotted">…</span>`), que algunos visores de Markdown no interpretan. En ese caso la clave ajena se identifica por las líneas de referencia (`tabla.columna` → `tabla.columna`) que figuran bajo cada tabla, donde aparecen todas las claves ajenas del documento.
>
> **Nota.** Cuando una clave ajena forma parte de la clave primaria se muestra en negrita y subrayado con `#` (es primaria) y su condición de ajena se indica en la línea de referencia que figura bajo la tabla.

Reglas de integridad referencial de las claves ajenas

Bajo cada tabla se indican las claves ajenas con el formato:

> `tabla_origen.columna_ajena` → `tabla_destino.columna_referenciada` (*política de nulos*, *política de borrado*, *política de actualización*)

- **Política de valores nulos:** `NOT NULL` o `NULL`.
- **Política de borrado** (qué ocurre en la tabla hija al borrar la fila padre):
  - `B:N` → anulación: la clave ajena pasa a `NULL` (`ON DELETE SET NULL`). Solo es posible si la clave ajena admite `NULL`.
  - `B:R` → restringido: se impide el borrado (`ON DELETE RESTRICT` / `NO ACTION`).
  - `B:C` → en cascada: se borran también las filas hijas (`ON DELETE CASCADE`).

    - `B:U` → valor por defecto: la clave ajena toma su valor por defecto (`ON DELETE SET DEFAULT`).
- **Política de actualización** (qué ocurre en la tabla hija al modificar la clave primaria referenciada): `M:N` (`ON UPDATE SET NULL`), `M:R` (`RESTRICT`), `M:C` (`CASCADE`) y `M:U` (`SET DEFAULT`), con el mismo significado que las anteriores.

> **Aviso.** MySQL con InnoDB acepta `SET DEFAULT` en la sintaxis pero **no lo implementa** (lo rechaza o lo trata como `RESTRICT` según la versión). `B:U`/`M:U` se mantienen como opciones del modelo lógico, pero en MySQL conviene evitarlas. Si se usa `B:N`/`M:N`, la columna debe ser `NULL`; **nunca** pueden combinarse con `NOT NULL`.

> **Coherencia de la integridad referencial:**
>
> - Una clave ajena que forma parte de la clave primaria es siempre `NOT NULL`, por lo que no admite `B:N` ni `M:N`.
> - Una clave ajena con `NOT NULL` y una política `B:C` o `B:R` es la combinación habitual de la participación obligatoria.
> - La columna referenciada debe ser clave primaria (o `UNIQUE`) de la tabla destino y tener el mismo tipo de dato.

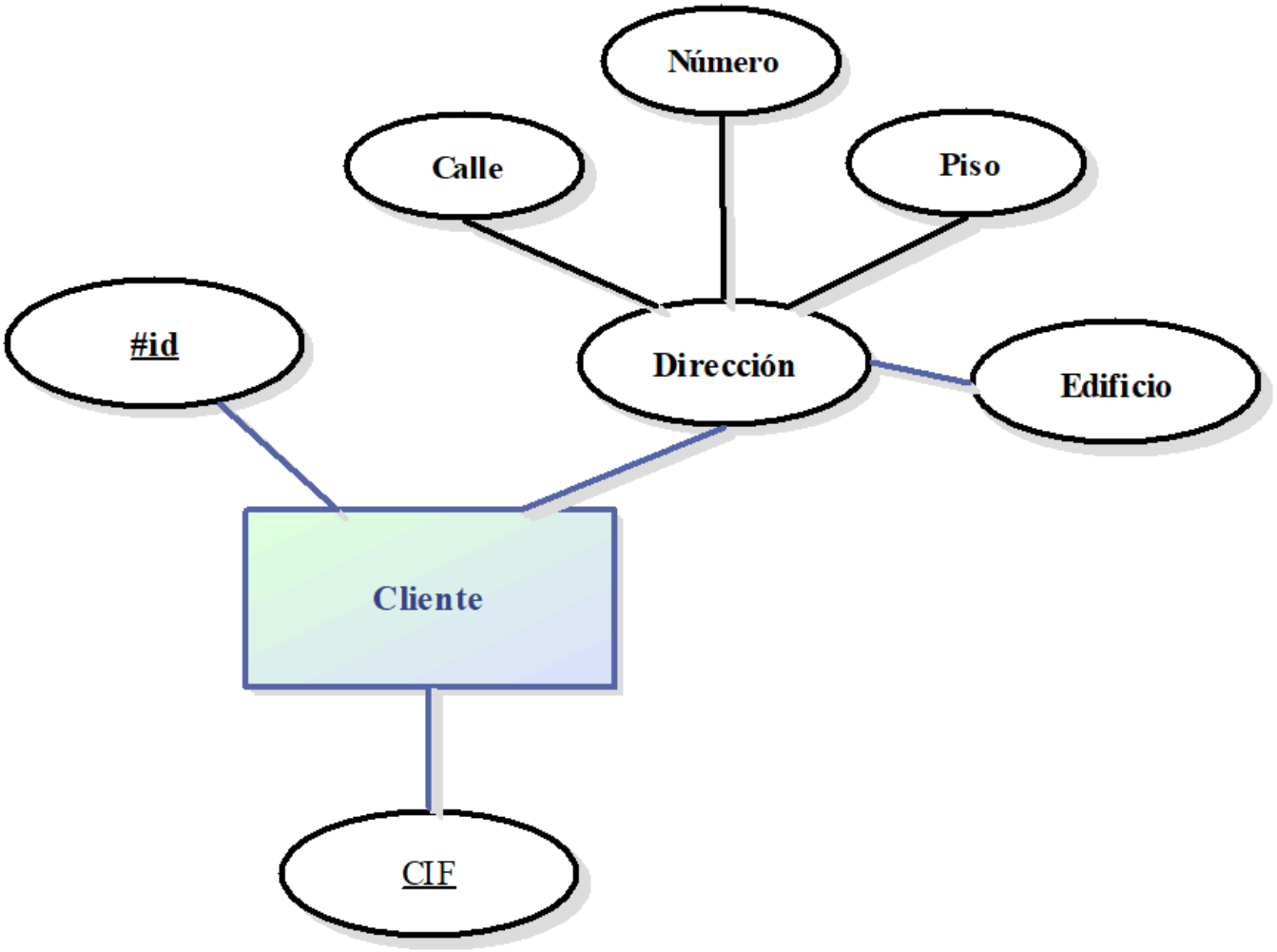
4.2.2 Transformación de atributos compuestos

Un atributo compuesto es aquel que puede dividirse en bloques de información más pequeños. El caso más común es el atributo *Domicilio*. Normalmente, en el domicilio de un cliente o de un proveedor no es necesario distinguir entre calle, número, portal, planta, etc. Si en nuestro diseño deseamos distinguir esos componentes, la única solución es crear atributos atómicos para cada uno de ellos. Esta situación se presenta en raras ocasiones, y normalmente el diseño se hace directamente con los atributos atómicos que se quieren distinguir.

La regla que aplicaremos es:

- El atributo compuesto **no** se incluye en la tabla, solamente los atributos que lo componen.

Veamos el ejemplo de la figura 4-3.



> Ilustración 4-3. Entidad Cliente con el atributo compuesto Domicilio (Calle, Número, Piso, Edificio).

La transformación al modelo relacional sería:

> Cliente(**#id**, CIF, Calle, Número, Piso, Edificio)

4.2.3 Transformación de los atributos multivaluados

El modelo relacional impone que todos los atributos sean **atómicos**, es decir, que almacenen un solo valor (primera forma normal). En el modelo E/R, en cambio, pueden representarse atributos multivaluados, que admiten más de un valor; por ejemplo, los teléfonos de un cliente que tiene varios.

Se aplica previamente la siguiente regla:

- Se crea una **entidad débil** con el nombre del atributo multivaluado y se relaciona mediante una **dependencia en identificación** con la entidad principal, que pasa a ser la entidad fuerte de la relación.

Veamos el siguiente ejemplo (figura 4-4).

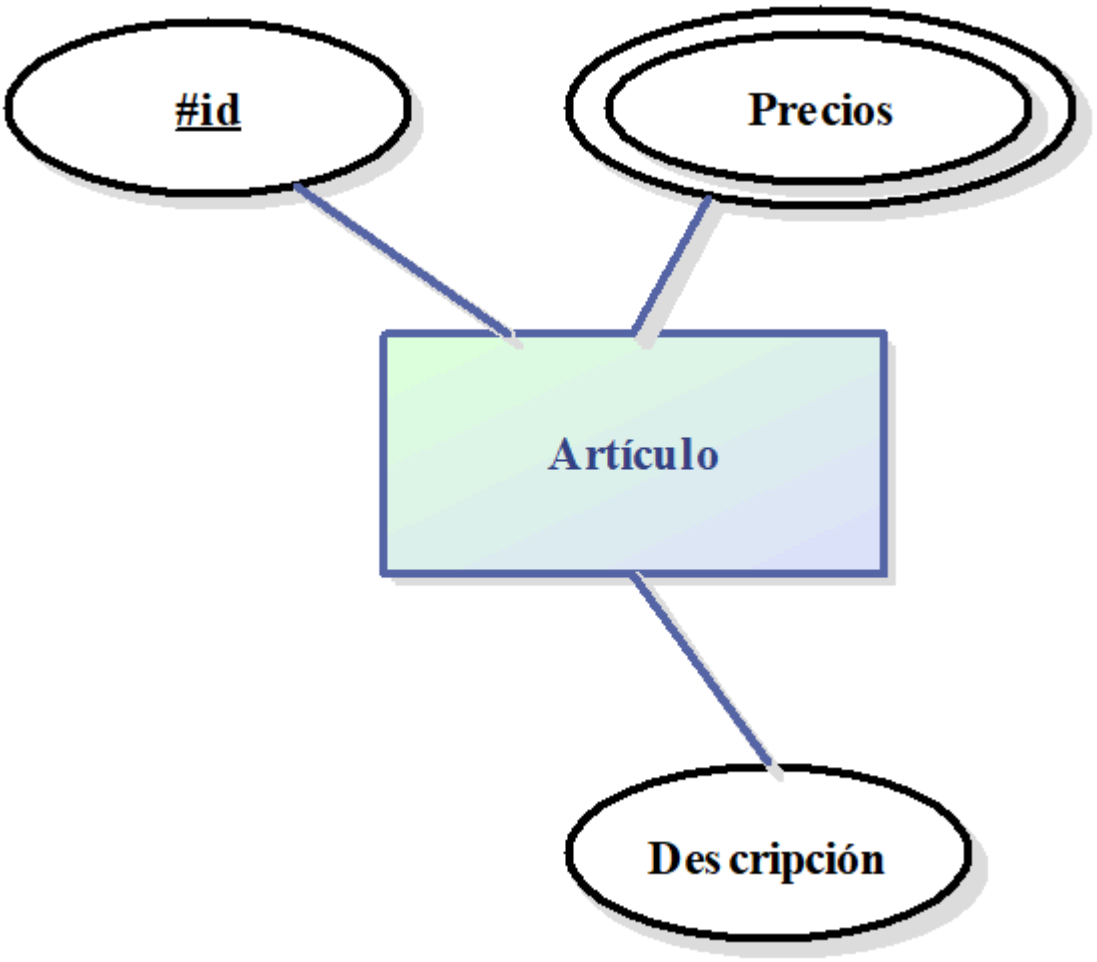


Ilustración 4-4. Entidad Artículos con el atributo multivaluado Precios.

Dado el modelo anterior, con el atributo multivaluado *Precios*, hay que crear una entidad débil *Precios*, relacionada con la entidad fuerte *Artículos* mediante una dependencia en identificación. El nuevo modelo es el de la figura 4-5.

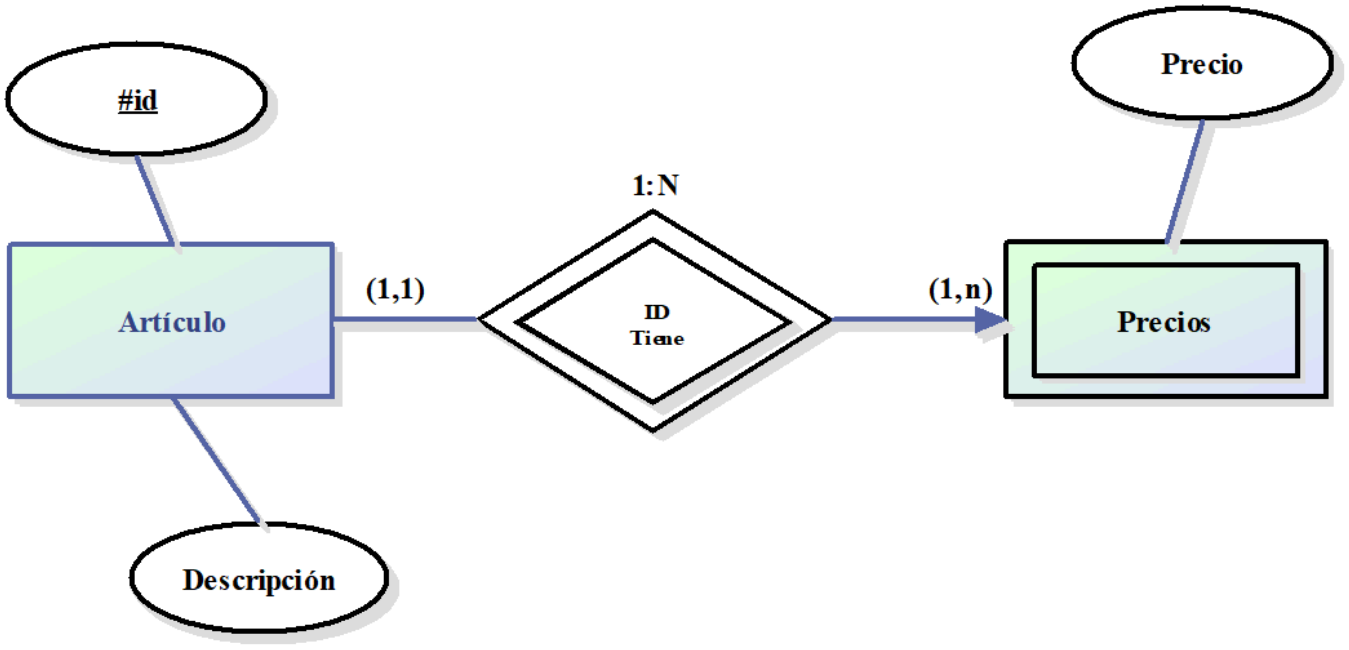


Ilustración 4-5. El atributo multivaluado Precios pasa a ser una entidad débil dependiente en identificación de Artículos.

Para la transformación al modelo relacional se crean las tablas *Articulo* y *Precios*. La clave primaria de *Precios* es la concatenación `id_articulo + Precio`, y `id_articulo` es además clave ajena que referencia a *Articulo*, con borrado y modificación en cascada.

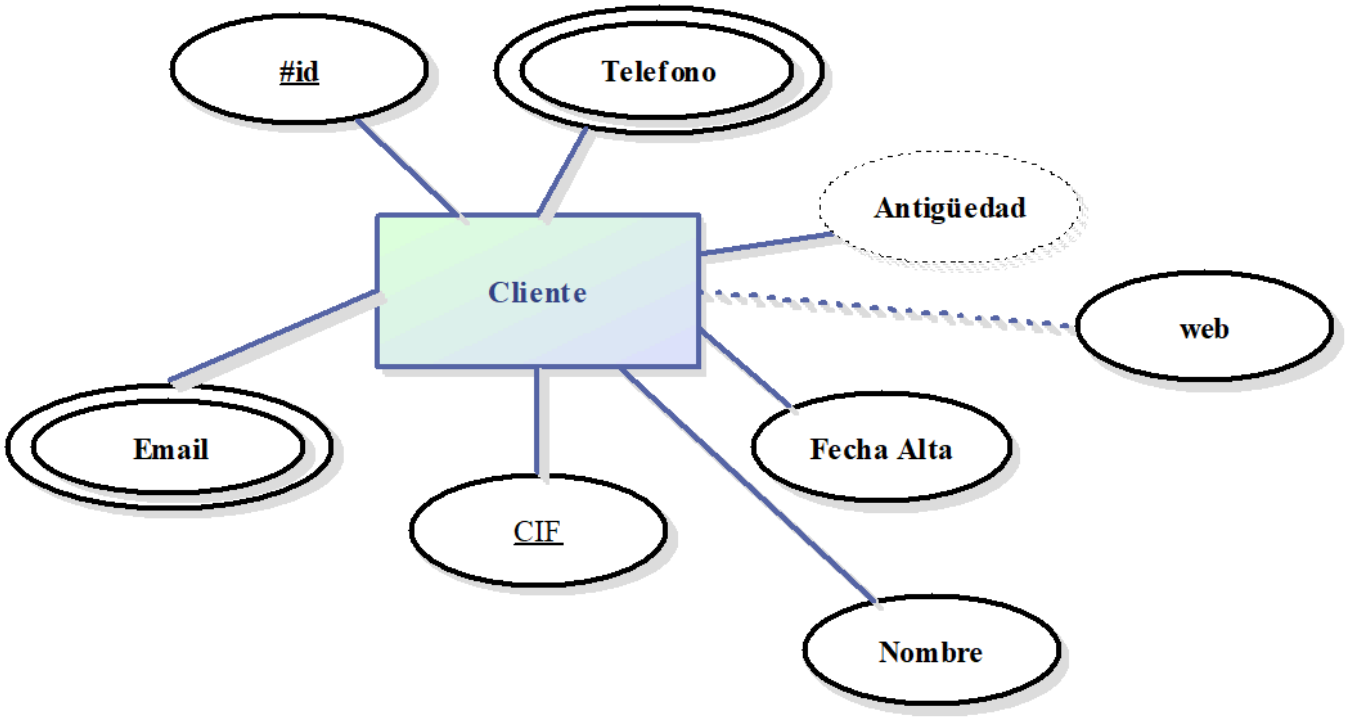
> Articulo(**#id**, Descripcion)
>
> Precios(**#id_articulo**, **#Precio**)
>
> - `Precios.id_articulo` → `Articulo.id` (NOT NULL, B:C, M:C)

En la implementación en SQL habrá que aplicar las siguientes restricciones:

- `PRIMARY KEY` en `id` de Articulo.
- `PRIMARY KEY` en `(id_articulo, Precio)` de Precios.
- `NOT NULL` en `Descripcion`.
- `FOREIGN KEY` de `id_articulo` en Precios, que referencia a Articulo (`ON DELETE CASCADE ON UPDATE CASCADE`).

4.2.4 Caso práctico I. Transformación de la entidad Cliente

En un diagrama entidad-relación nos encontramos con la siguiente entidad *Cliente* (figura 4-6).



> Ilustración 4-6. Cliente: id, CIF, Nombre, Web (opcional), Antigüedad (derivado), FechaAlta, Teléfonos (multivaluado), Email (multivaluado).

Aplicando las reglas de transformación de entidades y atributos obtenemos:

> Cliente(**#id**, CIF, Nombre, Web*, Antigüedad-d, FechaAlta)
>
> TelefonosClientes(**#id_cliente**, **#Telefono**)
>
> - `TelefonosClientes.id_cliente` → `Cliente.id` (NOT NULL, B:C, M:C)

> EmailCliente(**#id_cliente**, **#Email**)
>
> - `EmailCliente.id_cliente` → `Cliente.id` (NOT NULL, B:C, M:C)

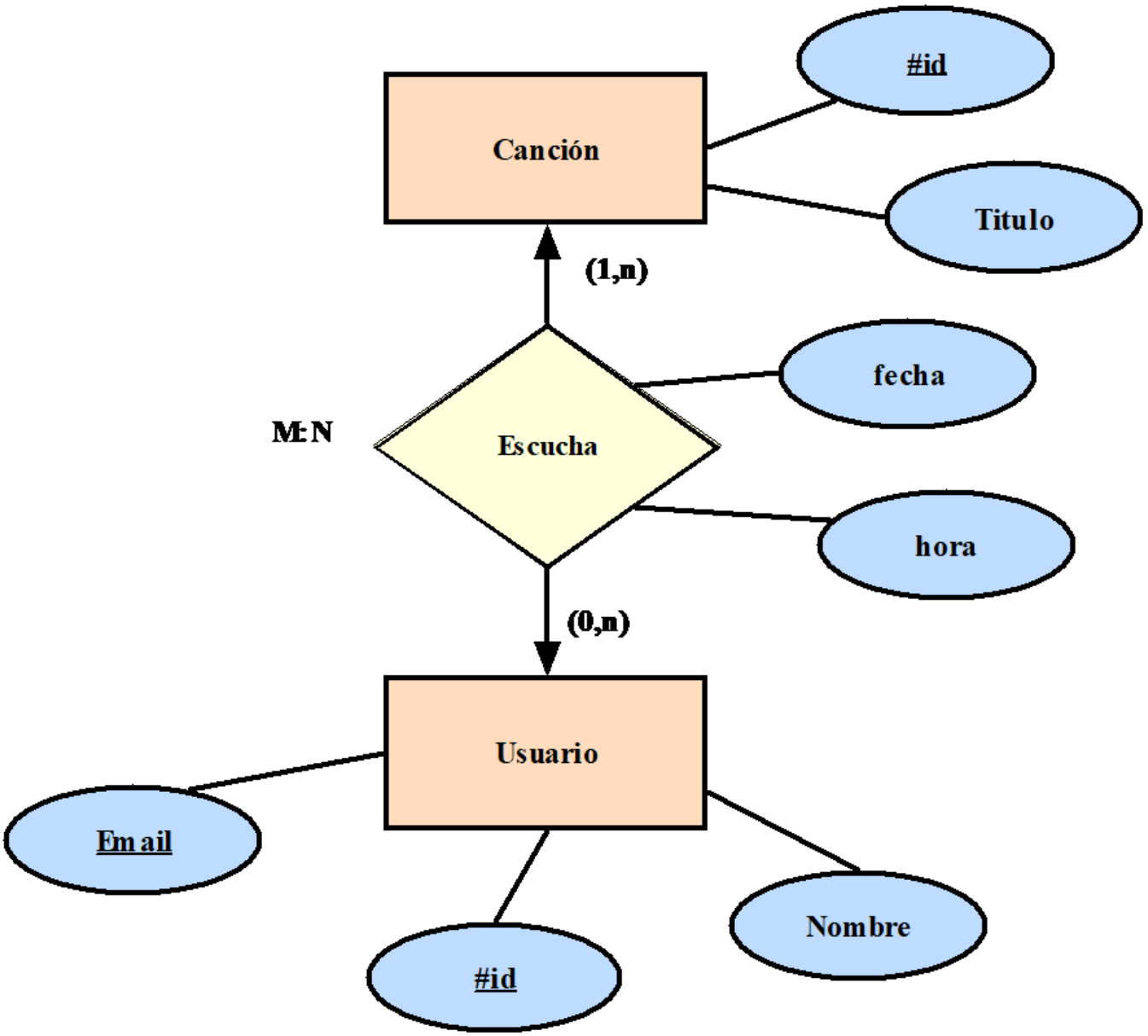
4.3 Transformación de las relaciones M:N

Es una de las reglas más sencillas, porque no hay casos excepcionales: este tipo de relación siempre implica la creación de una **nueva tabla**.

La regla dice:

- Una relación M:N se transforma en una **nueva tabla** cuya clave primaria es la **concatenación de las claves primarias** de las entidades que participan.
- En la nueva tabla se incluyen los atributos propios de la relación, si los tuviese.
- Cada uno de los atributos que forman la clave primaria es a su vez **clave ajena** que referencia a la tabla de la que procede.

Veamos el ejemplo de la figura 4-7.



> Ilustración 4-7. Usuario *escucha* Canción (M:N), con atributos Fecha y Hora.

El modelo relacional resultante es:

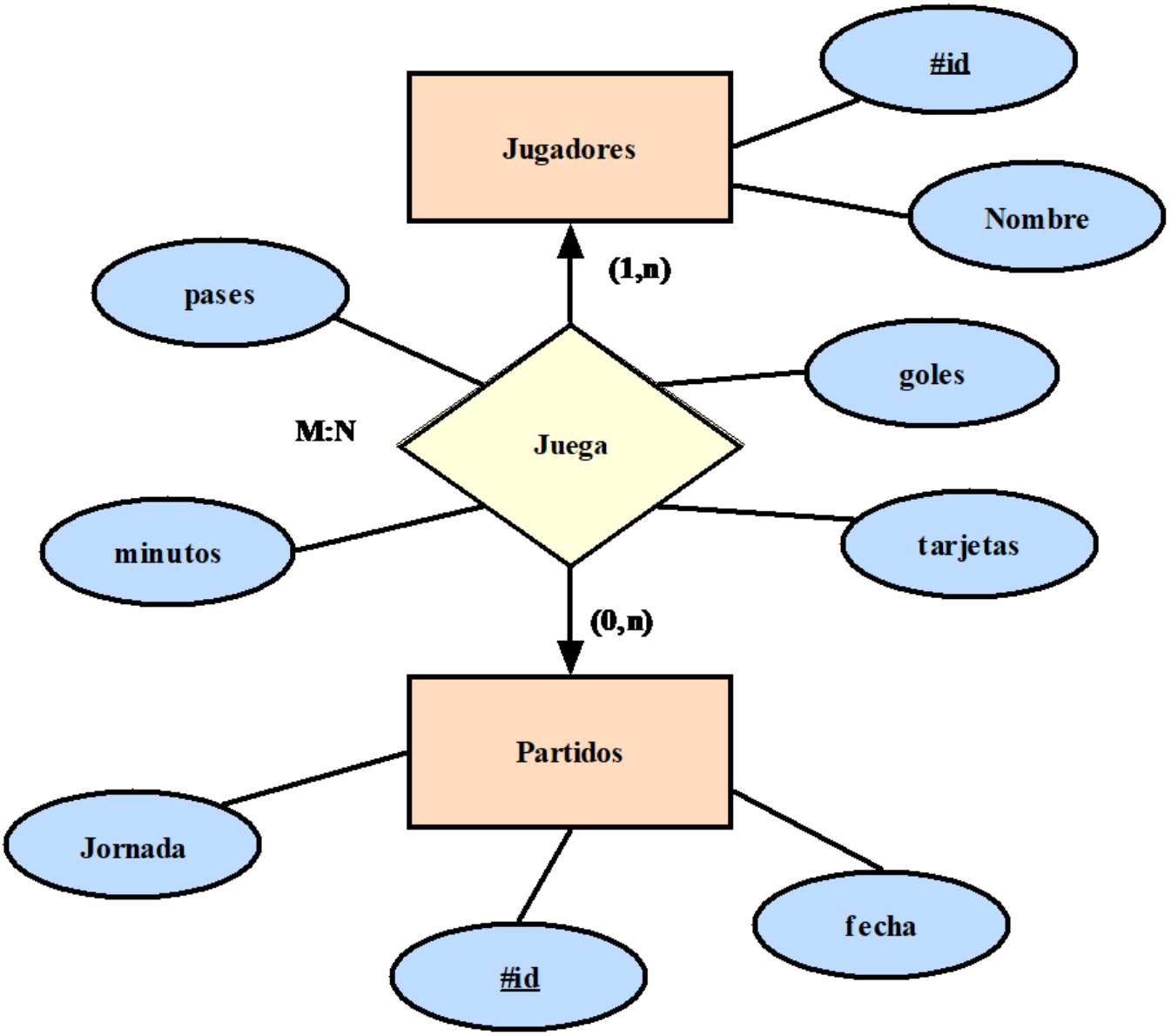
> Cancion(**#id**, Titulo)
>
> Usuario(**#id**, Nombre)
>
> Escucha(**#id_cancion**, **#id_usuario**, Fecha, Hora)
>
> - `Escucha.id_cancion` → `Cancion.id` (NOT NULL, B:R, M:C)
> - `Escucha.id_usuario` → `Usuario.id` (NOT NULL, B:R, M:C)

Si la relación *Escucha* no tuviese atributos propios:

> Escucha(**#id_cancion**, **#id_usuario**)
>
> - `Escucha.id_cancion` → `Cancion.id` (NOT NULL, B:R, M:C)
> - `Escucha.id_usuario` → `Usuario.id` (NOT NULL, B:R, M:C)

Observación. Con la regla general, la clave primaria `(id_cancion, id_usuario)` impide registrar que un mismo usuario escuche la misma canción más de una vez. Si el enunciado exige guardar todas las escuchas, hay que añadir a la clave primaria un atributo que distinga cada ocurrencia (por ejemplo `Fecha` y `Hora`) o crear una clave sustituta.

Veamos otro caso práctico.



> Jugadores *juegan* Partidos (M:N), con atributos Goles, Pases, Tarjetas y Minutos.

> Jugadores(**#id**, Nombre)
>
> Partidos(**#id**, Fecha, Jornada)
>
> Juegan(**#id_jugador**, **#id_partido**, Goles, Pases, Tarjetas, Minutos)
>
> - `Juegan.id_jugador` → `Jugadores.id` (NOT NULL, B:R, M:C)
> - `Juegan.id_partido` → `Partidos.id` (NOT NULL, B:R, M:C)

4.4 Transformación de las relaciones 1:N

Al transformar las relaciones 1:N hay que tener en cuenta la **cardinalidad mínima de la entidad que participa con cardinalidad máxima 1**. Pueden darse dos casos:

1. Que la cardinalidad mínima sea 1 (figura 4-8).
2. Que la cardinalidad mínima sea 0 (figura 4-9).

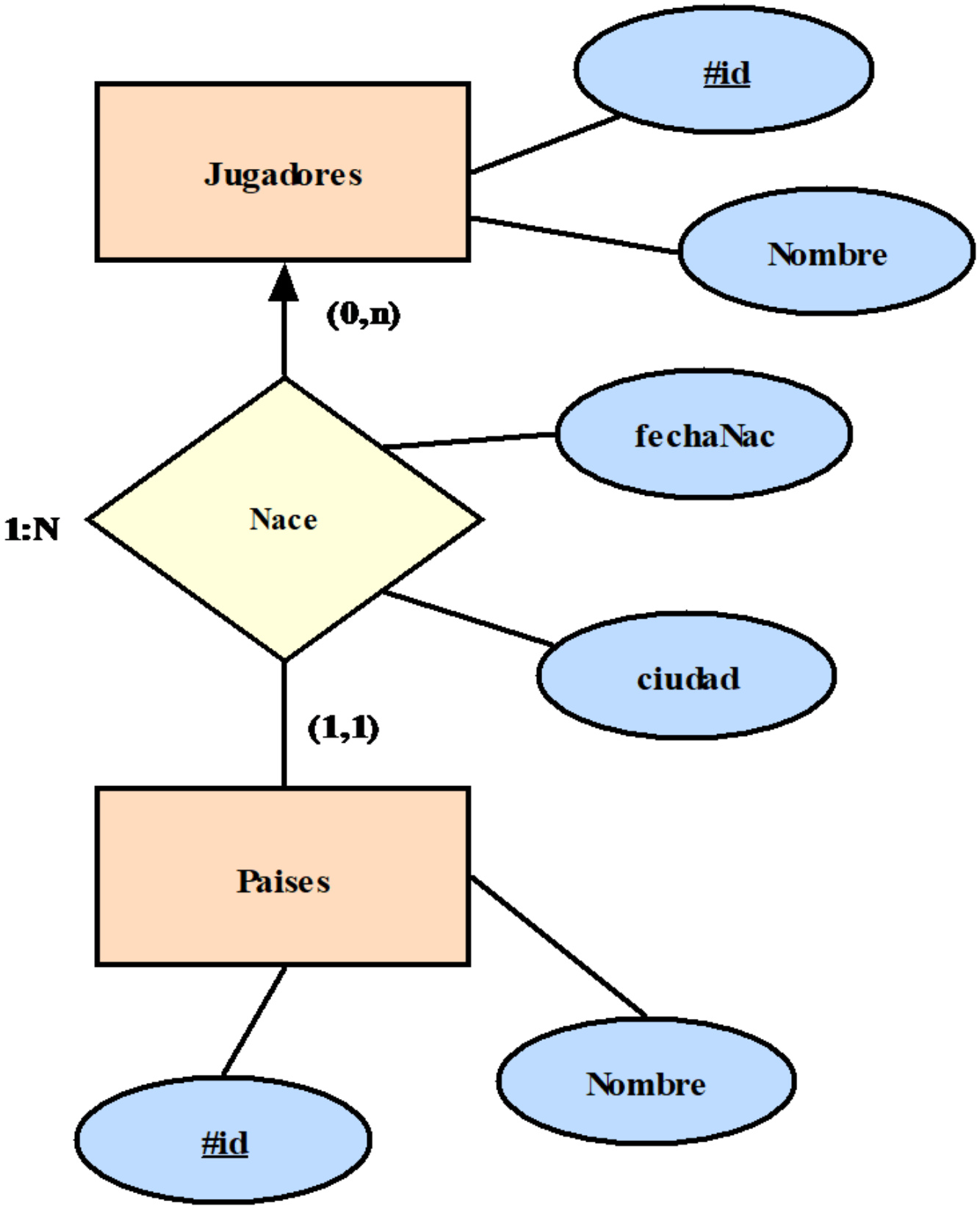
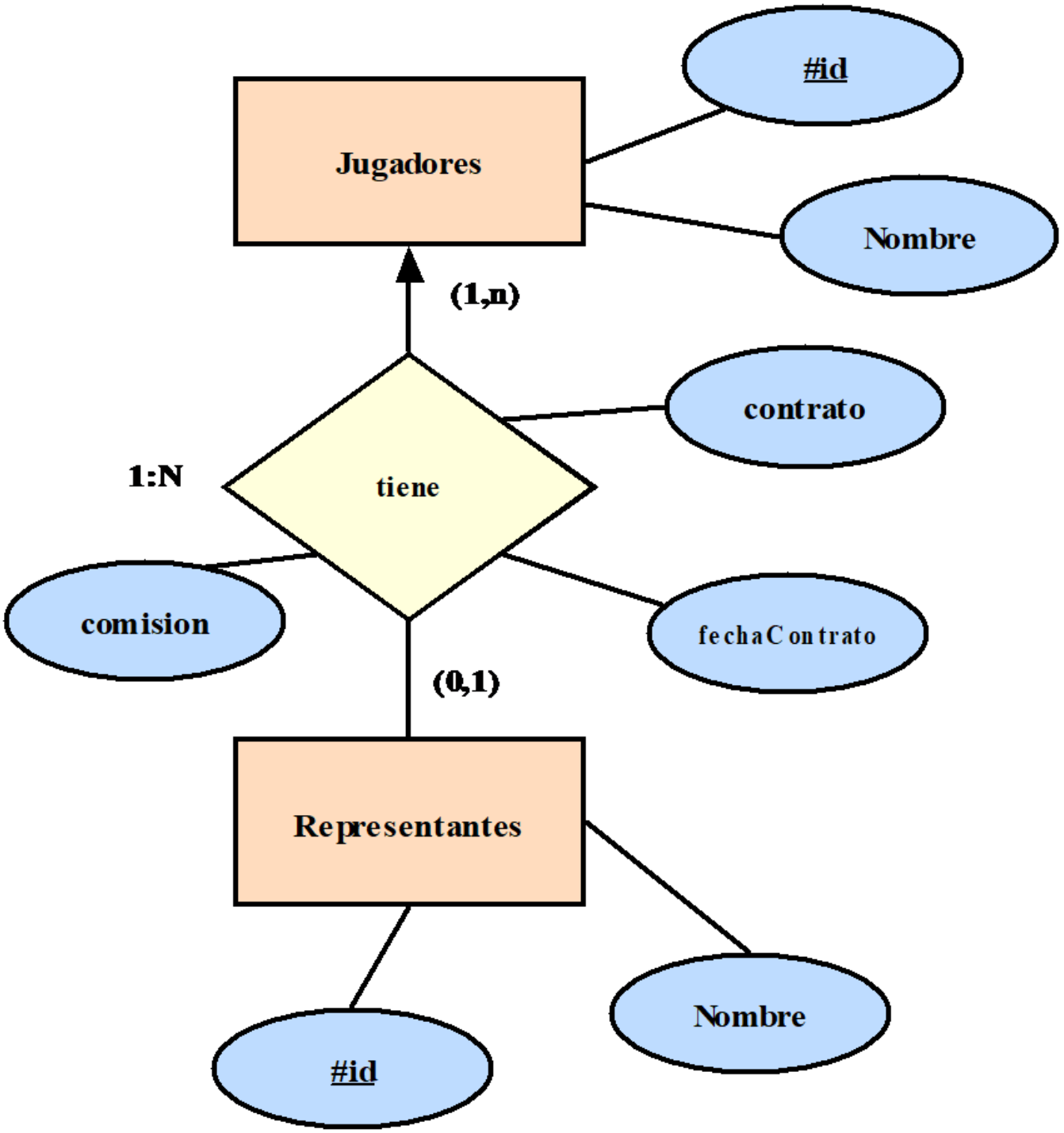


Ilustración 4-8. Todo jugador pertenece a un país: País (1,1) — Jugador (0,N).



> Ilustración 4-9. Es posible que haya jugadores que no tengan representante: Representante (0,1) — Jugador (0,N).

4.4.1 Caso A. (1,1)

Si la entidad que participa con cardinalidad máxima 1 tiene cardinalidad mínima 1:

- Se **propaga la clave primaria** de la entidad del lado 1 a la tabla del lado N.
- La tabla del lado N dispone de un nuevo atributo sobre el que se aplica `FOREIGN KEY` que referencia a la tabla del lado 1. Al ser la participación obligatoria, ese atributo es `NOT NULL`.
- Si la relación tiene atributos, se propagan también a la tabla del lado N.

Aplicando las reglas a la figura 4-8:

> Paises(**#id**, Nombre)
>
> Jugadores(**#id**, Nombre, FecNacimiento, Ciudad, id_pais)
>
> - `Jugadores.id_pais` → `Paises.id` (NOT NULL, B:R, M:C)

4.4.2 Caso B. (0,1)

Si la entidad que participa con cardinalidad máxima 1 tiene cardinalidad mínima 0, se evitan los valores nulos en la tabla del lado N así:

- Se crea una **nueva tabla** cuya clave primaria es la clave primaria de la entidad del lado N, que es al mismo tiempo clave ajena a la tabla de la que procede.
- La nueva tabla incluye también, como clave ajena, la clave primaria de la entidad del lado 1.
- Si la relación 1:N tiene atributos, se incluyen también en la nueva tabla.

Aplicando las reglas a la figura 4-9:

> Representante(**#id**, Nombre)
>
> Jugadores(**#id**, Nombre)
>
> TieneRe(**#id_jugador**, id_representante, FechaContratacion, Comision, Importe)
>
> - `TieneRe.id_jugador` → `Jugadores.id` (NOT NULL, B:C, M:C)
> - `TieneRe.id_representante` → `Representante.id` (NOT NULL, B:C, M:C)

Observación. Una alternativa habitual es no crear la tabla nueva y propagar la clave del lado 1 a la tabla del lado N permitiendo `NULL` (`id_representante*` en Jugadores, con `B:N`, `M:C`). Es más simple, pero introduce valores nulos; este tema aplica la solución de la nueva tabla.

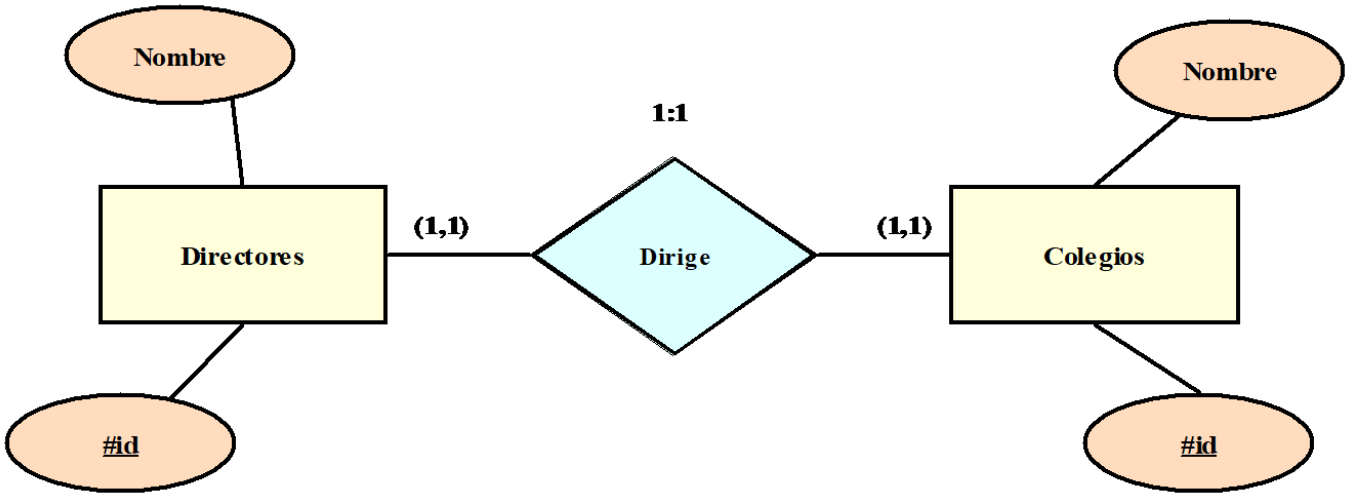
4.5 Transformación de las relaciones 1:1

La transformación de las relaciones 1:1 es la más compleja, porque pueden darse tres casos:

1. Ambas entidades con cardinalidad (1,1).
2. Ambas entidades con cardinalidad (0,1).
3. Una entidad con (1,1) y la otra con (0,1).

4.5.1 Caso A. E1 (1,1) y E2 (1,1)

- Se propaga la clave de **cualquiera** de las dos entidades a la tabla de la otra, como clave ajena. Para garantizar que la relación sea 1:1, esa clave ajena debe ser además `UNIQUE` (y `NOT NULL`).
- La elección de qué clave propagar depende del diseñador y de la semántica de los requisitos.



> Ilustración 4-10. Director (1,1) — Colegio (1,1).

Modelo relacional resultante:

> Directores(**#id**, Nombre)
>
> Colegios(**#id**, Nombre, id_director)
>
> - `Colegios.id_director` → `Directores.id` (NOT NULL, B:R, M:C), además `UNIQUE`

También sería válida la otra opción:

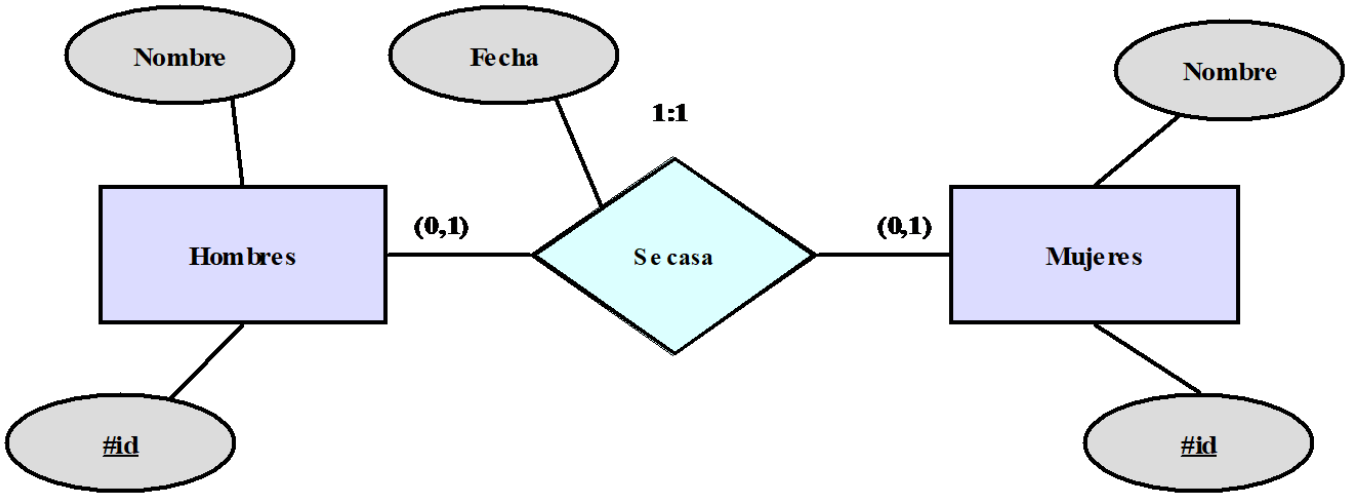
> Directores(**#id**, Nombre, id_colegio)
>
> - `Directores.id_colegio` → `Colegios.id` (NOT NULL, B:R, M:C), además `UNIQUE`
>
> Colegios(**#id**, Nombre)

> Nota: al ser ambas participaciones obligatorias, en SQL la inserción simultánea de ambas filas requiere diferir la comprobación (o fusionar ambas tablas en una sola; ver 4.12).

4.5.2 Caso B. E1 (0,1) y E2 (0,1)

- Se crea una **nueva tabla** cuya clave primaria es la clave primaria de una de las entidades; la clave primaria de la otra entidad pasa a la nueva tabla como clave ajena, que además debe ser `UNIQUE` y `NOT NULL`.
- En la nueva tabla se incluyen los atributos propios de la relación 1:1, si los tuviese.

Estas reglas no son idénticas a las de las relaciones M:N, pero son muy parecidas.



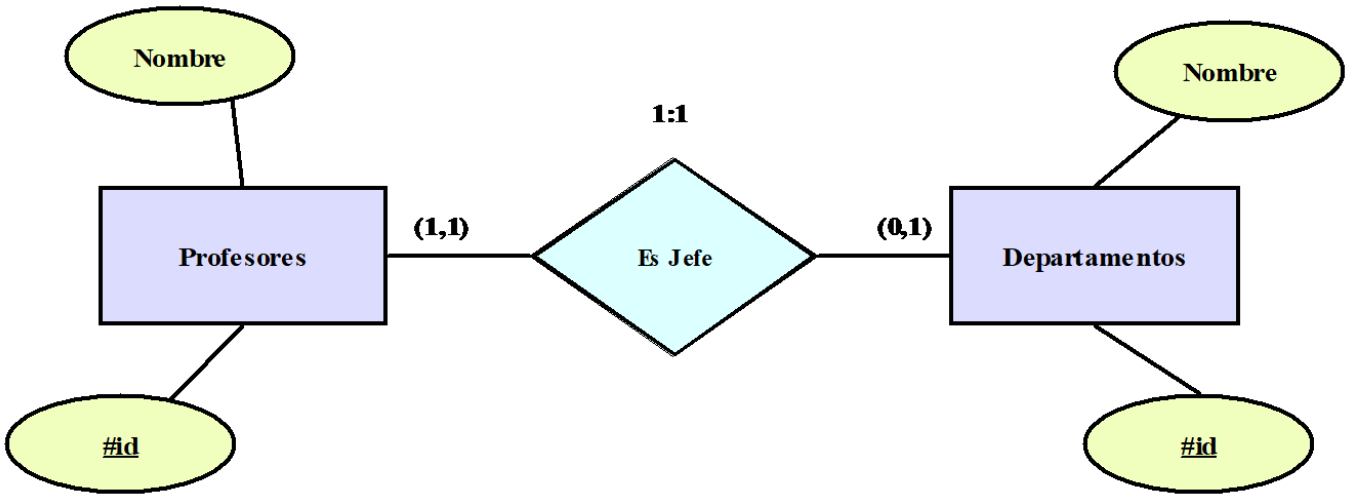
> Ilustración 4-11. Hombre (0,1) — *EsEsposo* — Mujer (0,1).

> Hombres(**#id**, Nombre)
>
> Mujeres(**#id**, Nombre)
>
> EsEsposo(**#id_hombre**, id_mujer, Fecha)
>
> - `EsEsposo.id_hombre` → `Hombres.id` (NOT NULL, B:C, M:C)
> - `EsEsposo.id_mujer` → `Mujeres.id` (NOT NULL, B:C, M:C), además `UNIQUE`

4.5.3 Caso C. E1 (0,1) y E2 (1,1)

- Se propaga la clave de la entidad con cardinalidad (0,1) a la tabla de la entidad con cardinalidad (1,1) (la de participación obligatoria), de modo que la clave ajena sea `NOT NULL` y `UNIQUE`. De esta forma se evitan los valores nulos.
- Si la relación 1:1 tiene atributos, se propagan también a la tabla de la entidad (1,1).

Ejemplo (figura 4-12): todo departamento tiene un profesor que lo dirige, pero no todo profesor dirige un departamento.



> Ilustración 4-12. Profesor (0,1) — *dirige* — Departamento (1,1).

> Profesores(**#id**, Nombre)

> Departamento(**#id**, Nombre, id_profesor)
>
> - `Departamento.id_profesor` → `Profesores.id` (NOT NULL, B:R, M:C), además `UNIQUE`

4.6 Transformación de las relaciones ternarias

Una relación ternaria es aquella en la que participan tres entidades. Según su tipo de correspondencia puede ser 1:1:1, M:1:1, M:N:1 o M:N:P.

Regla general:

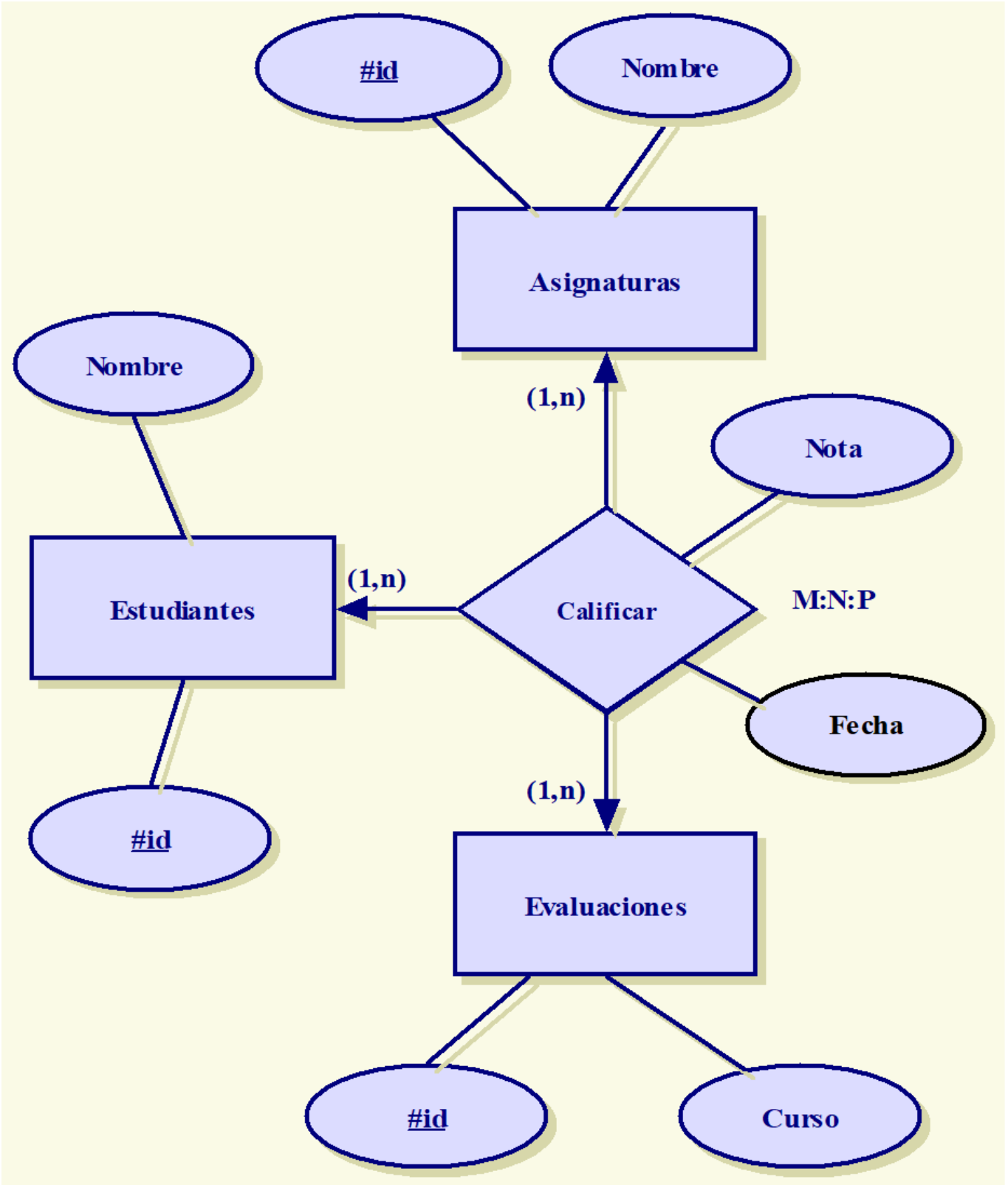
- Se crea una **nueva tabla** que tendrá como atributos las claves primarias de las tres entidades relacionadas.
- Si la relación tiene atributos, se incluyen también en la nueva tabla.
- La clave primaria de la nueva tabla depende de la correspondencia:

Tipo	Clave primaria de la nueva tabla
M:N:P	Concatenación de las claves primarias de las tres entidades.
M:N:1	Concatenación de las claves de las entidades con cardinalidad M y N.
M:1:1	Clave de la entidad con cardinalidad M + la de una cualquiera de las dos con cardinalidad 1.
1:1:1	Concatenación de las claves de dos cualesquiera de las tres entidades.

4.6.1 Caso A. M:N:P

La clave primaria es la concatenación de las claves primarias de las tres entidades.

Ejemplo (figura 4-13): a un estudiante se le asigna una calificación en una asignatura correspondiente a una determinada evaluación.



Estudiantes(**#id**, Nombre)

Asignaturas(**#id**, Nombre)

Evaluaciones(**#id**, Curso)

Calificar(**#id_estudiante**, **#id_asignatura**, **#id_eval**, Fecha, Nota)

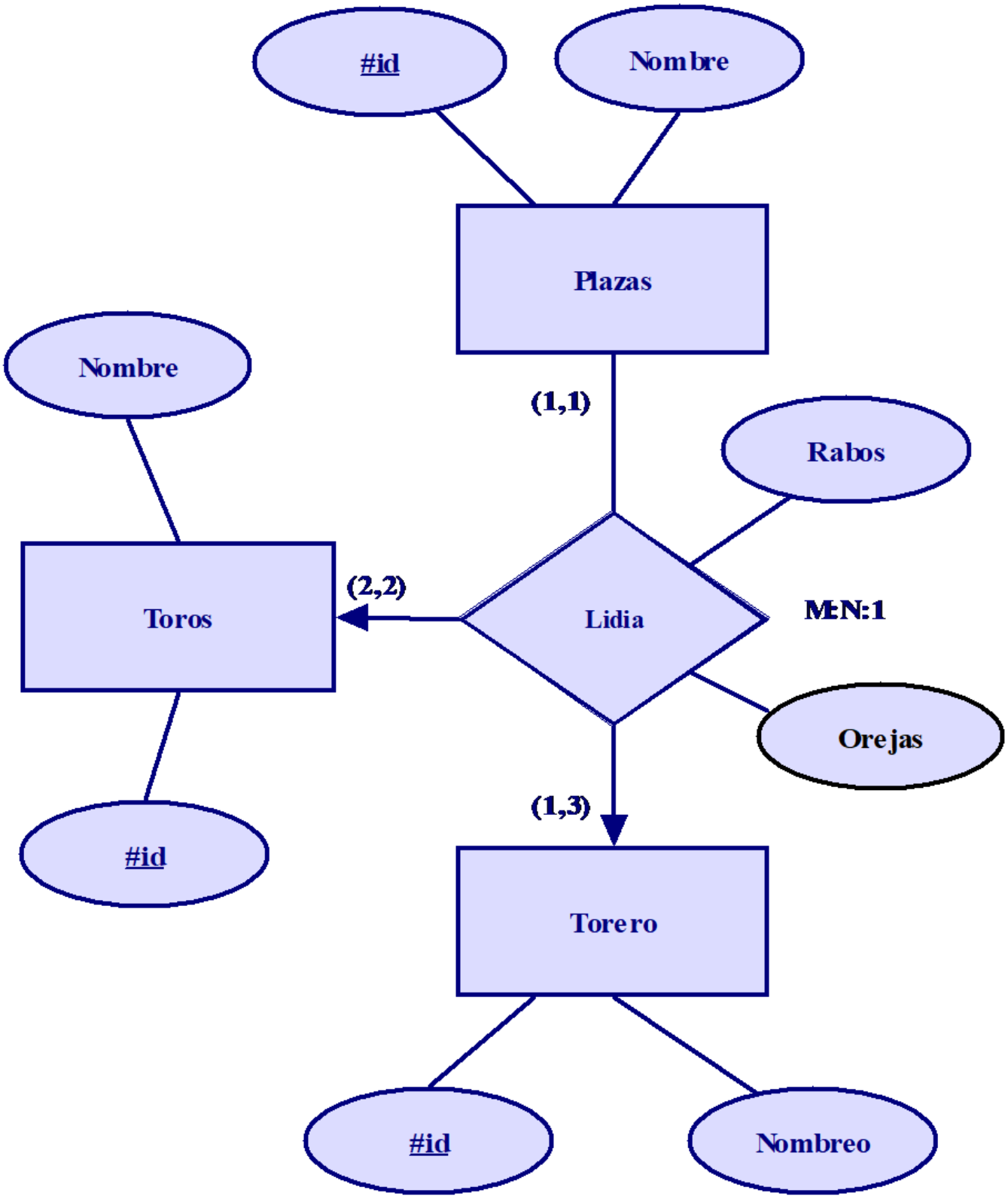
- `Calificar.id_estudiante` → `Estudiantes.id` (NOT NULL, B:R, M:C)
- `Calificar.id_asignatura` → `Asignaturas.id` (NOT NULL, B:R, M:C)

- `Calificar.id_eval` → `Evaluaciones.id` (NOT NULL, B:R, M:C)

4.6.2 Caso B. M:N:1

La clave primaria es la concatenación de las claves de las entidades con cardinalidad M y N.

Ejemplo (figura 4-14): los toros son lidiados por toreros en una plaza.



Plazas(**#id**, Nombre)

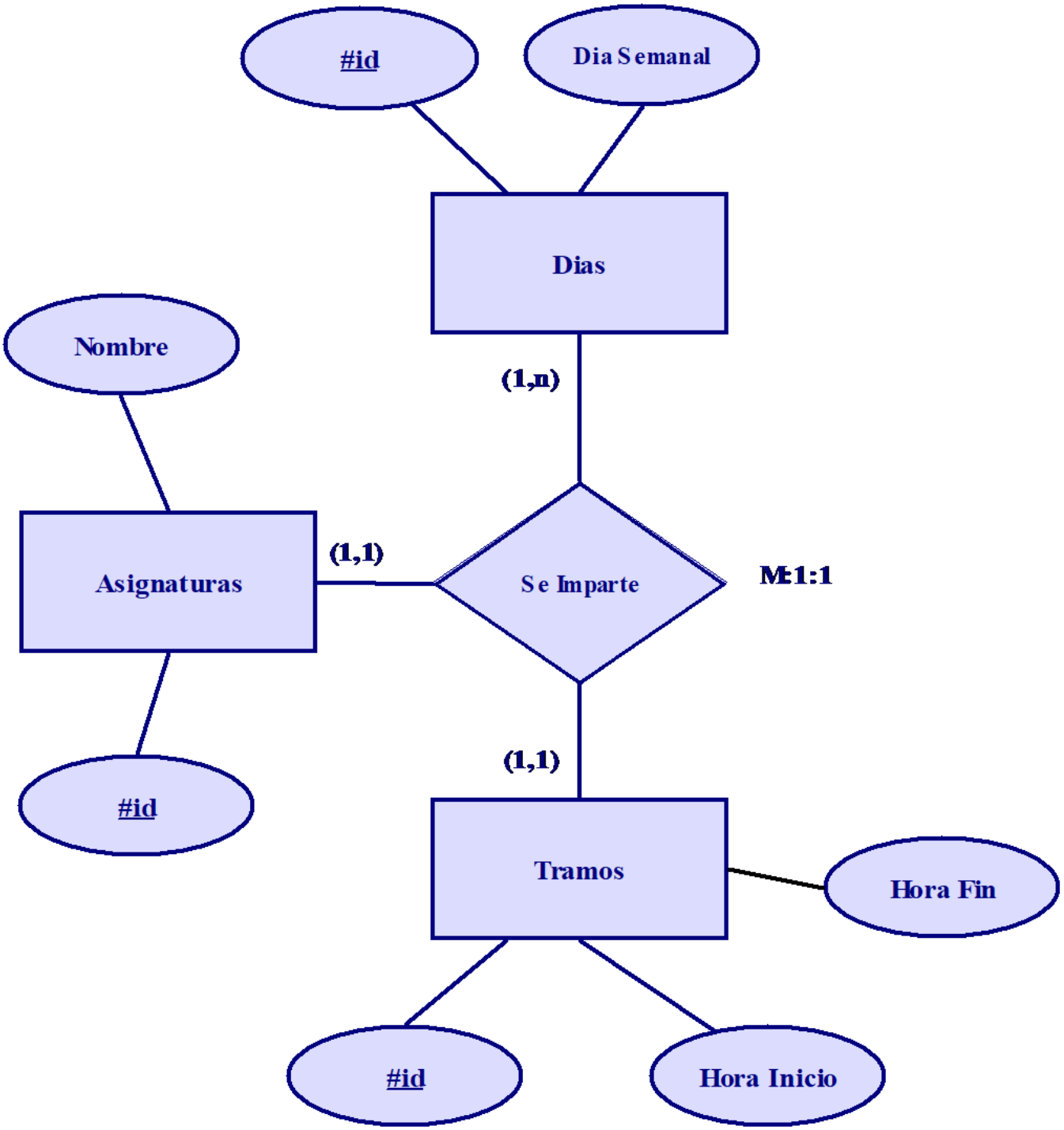
> Toros(**#id**, Nombre)
>
> Toreros(**#id**, Nombre)
>
> Lidia(**#id_torero**, **#id_toro**, id_plaza, Rabos, Orejas)
>
> - `Lidia.id_torero` → `Toreros.id` (NOT NULL, B:R, M:C)
> - `Lidia.id_toro` → `Toros.id` (NOT NULL, B:R, M:C)
> - `Lidia.id_plaza` → `Plazas.id` (NOT NULL, B:R, M:C)

No es necesario incluir `id_plaza` en la clave primaria de *Lidia*: para cada pareja (torero, toro) la plaza queda determinada.

4.6.3 Caso C. M:1:1

La clave primaria es la concatenación de la clave de la entidad con cardinalidad M y la de cualquiera de las dos entidades con cardinalidad 1.

Ejemplo (figura 4-15): modelo para la confección de horarios en un centro educativo.



Pueden obtenerse dos soluciones igualmente válidas.

Opción A:

> Dias(**#id**, DiaSemanal)
>
> Asignaturas(**#id**, Nombre)
>
> Tramos(**#id**, HoraInicio, HoraFin)
>
> Impartir(**#id_dia**, **#id_asignatura**, id_tramo)
>
> - `Impartir.id_dia` → `Dias.id` (NOT NULL, B:R, M:C)
> - `Impartir.id_asignatura` → `Asignaturas.id` (NOT NULL, B:R, M:C)
> - `Impartir.id_tramo` → `Tramos.id` (NOT NULL, B:R, M:C)

Opción B:

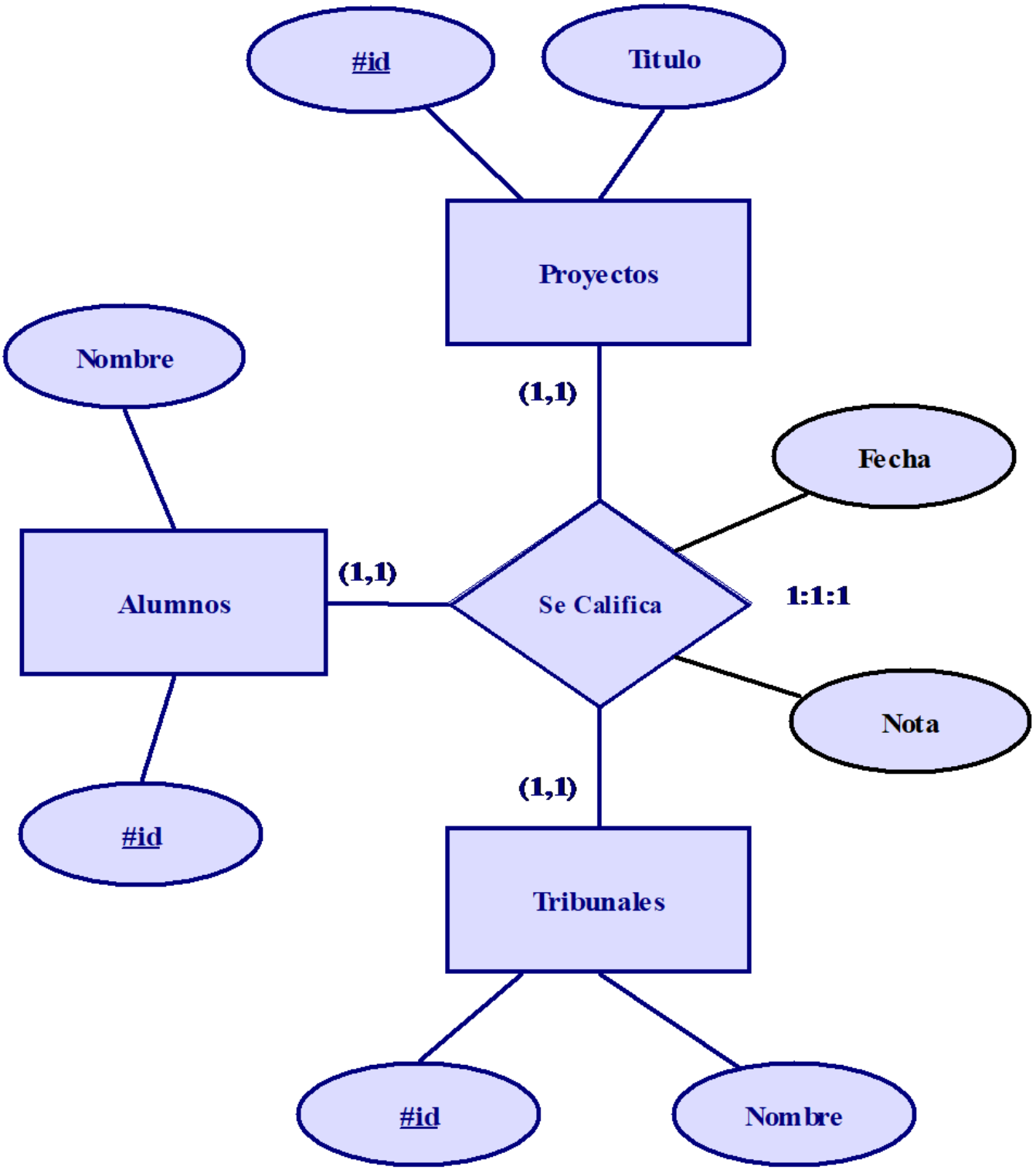
> Impartir(**#id_dia**, **#id_tramo**, id_asignatura)
>
> - `Impartir.id_dia` → `Dias.id` (NOT NULL, B:R, M:C)
> - `Impartir.id_tramo` → `Tramos.id` (NOT NULL, B:R, M:C)
> - `Impartir.id_asignatura` → `Asignaturas.id` (NOT NULL, B:R, M:C)

(Las tablas Dias, Asignaturas y Tramos son las mismas que en la opción A.)

4.6.4 Caso D. 1:1:1

La clave primaria es la concatenación de las claves de dos cualesquiera de las tres entidades.

Ejemplo (figura 4-16).



Alumnos(**#id**, Nombre)

Proyectos(**#id**, Titulo)

Tribunales(**#id**, Nombre)

Calificar(**#id_alumno**, **#id_proyecto**, id_tribunal, Fecha, Nota)

- `Calificar.id_alumno` → `Alumnos.id` (NOT NULL, B:R, M:C)
- `Calificar.id_proyecto` → `Proyectos.id` (NOT NULL, B:R, M:C)
- `Calificar.id_tribunal` → `Tribunales.id` (NOT NULL, B:R, M:C)

También serían válidas estas otras claves primarias para *Calificar*:

- `Calificar(#id_alumno, #id_tribunal, id_proyecto, Fecha, Nota)`
- `Calificar(#id_proyecto, #id_tribunal, id_alumno, Fecha, Nota)`

En cualquier caso, la clave ajena que queda fuera de la clave primaria debe declararse `NOT NULL` y, si la correspondencia lo exige, `UNIQUE`.

4.7 Transformación de las relaciones n-arias

La transformación de las relaciones n-arias es una generalización de las ternarias. En todos los casos se obtiene una **nueva tabla** con las claves primarias de las n entidades y todos los atributos de la relación. Se distinguen:

- **a)** Si todas las entidades están conectadas con «muchos», la clave primaria de la nueva tabla es la concatenación de las claves de las n entidades.
- **b)** Si una o más entidades están conectadas con «uno», la clave primaria está formada por las claves de $n - 1$ entidades, con la condición de que la entidad cuya clave no se incluye sea una de las conectadas con «uno».

4.8 Transformación de las relaciones reflexivas

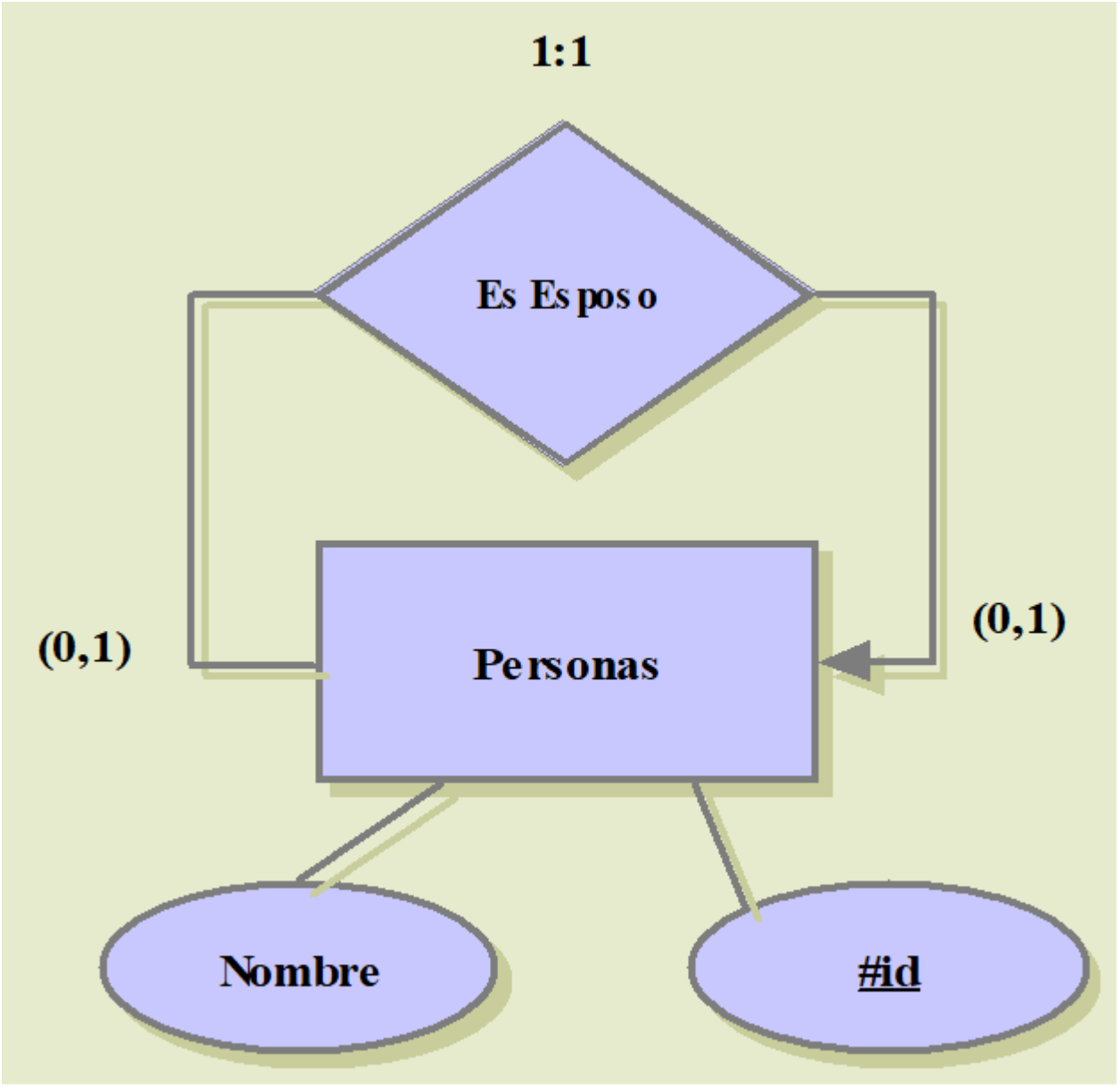
Las relaciones recursivas, reflexivas o unarias se transforman de forma similar al resto. En general:

- Si la relación reflexiva es **1:1 con (0,1) y (0,1)**, se crea una **nueva tabla** cuya clave primaria es la clave de la entidad; la nueva tabla incluye además la clave de la entidad propagada de nuevo con otro nombre, como clave ajena.
- Si la relación reflexiva es **1:N** (o 1:1 en los casos no contemplados en el punto anterior), se propaga la clave de la entidad hacia sí misma con un nombre distinto, de modo que pasa a ser clave ajena que referencia a su propia tabla.
- Si la relación reflexiva es **M:N**, se crea una **nueva tabla** cuya clave primaria es la concatenación doble de la clave de la entidad. Como en el modelo relacional no pueden existir dos atributos con el mismo nombre en una tabla, hay que renombrar uno de ellos.

En todos los casos, si la relación tiene atributos, se incluyen en la tabla correspondiente.

4.8.1 Caso A. 1:1 con (0,1) y (0,1)

Figura 4-17.



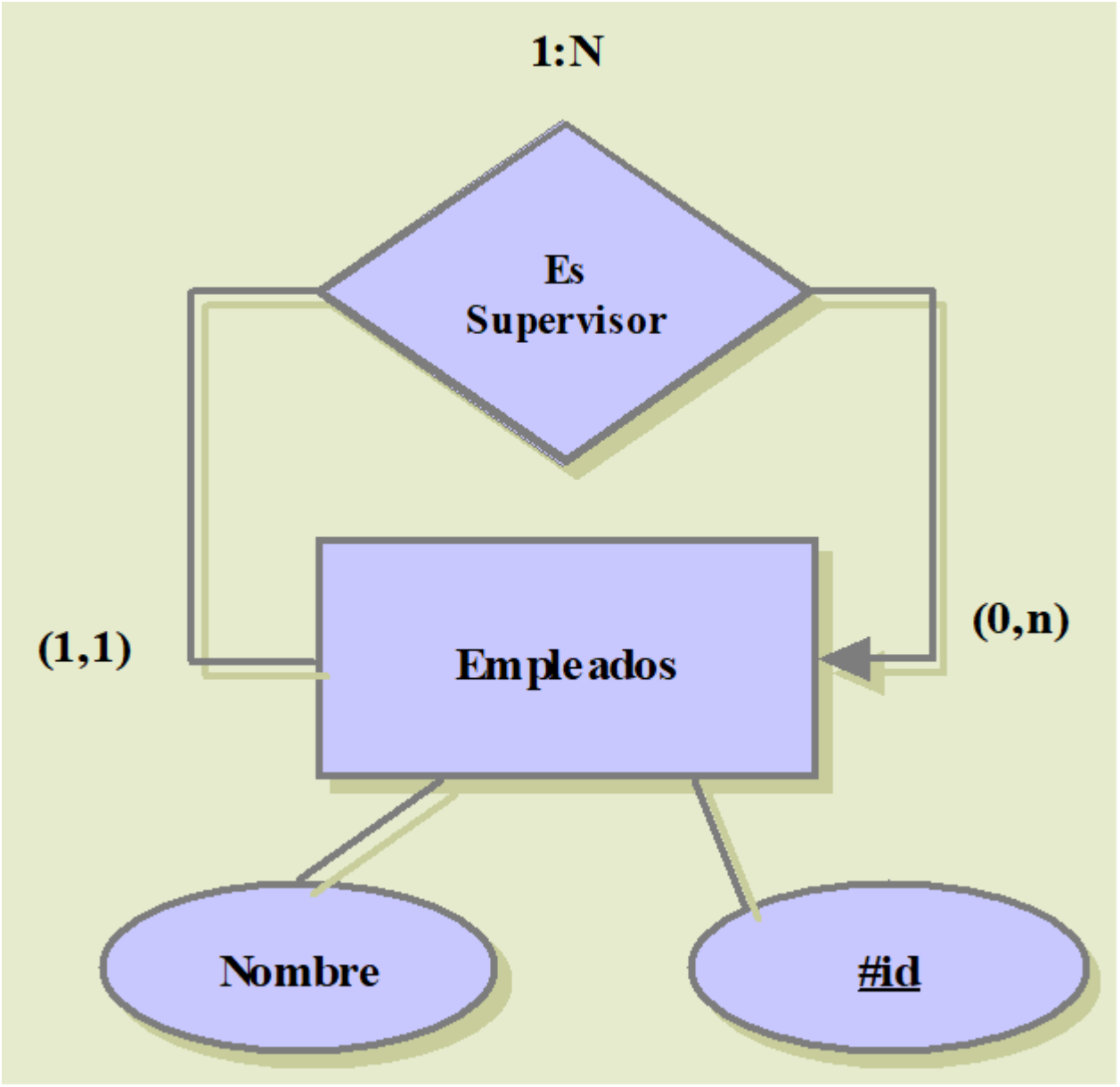
La relación es recursiva y de tipo 1:1, con participación opcional en ambos extremos. Se crea una nueva tabla *EsEsposo* cuya clave primaria es la de *Personas* y que incluye otra clave ajena, `id_conyuge`, que referencia a la misma tabla *Personas* (`UNIQUE` y `NOT NULL`):

> Personas(**#id**, Nombre)
>
> EsEsposo(**#id_persona**, id_conyuge)
>
> - `EsEsposo.id_persona` → `Personas.id` (NOT NULL, B:C, M:C)
> - `EsEsposo.id_conyuge` → `Personas.id` (NOT NULL, B:C, M:C), además `UNIQUE`

> Restricción semántica no expresable en el modelo (criterio RA6.h): una persona no puede ser su propio cónyuge (`CHECK (id_persona <> id_conyuge)`) y la relación es simétrica; hay que documentarlo y, si se desea, controlarlo con un trigger.

4.8.2 Caso B. 1:N o 1:1

Figura 4-18.

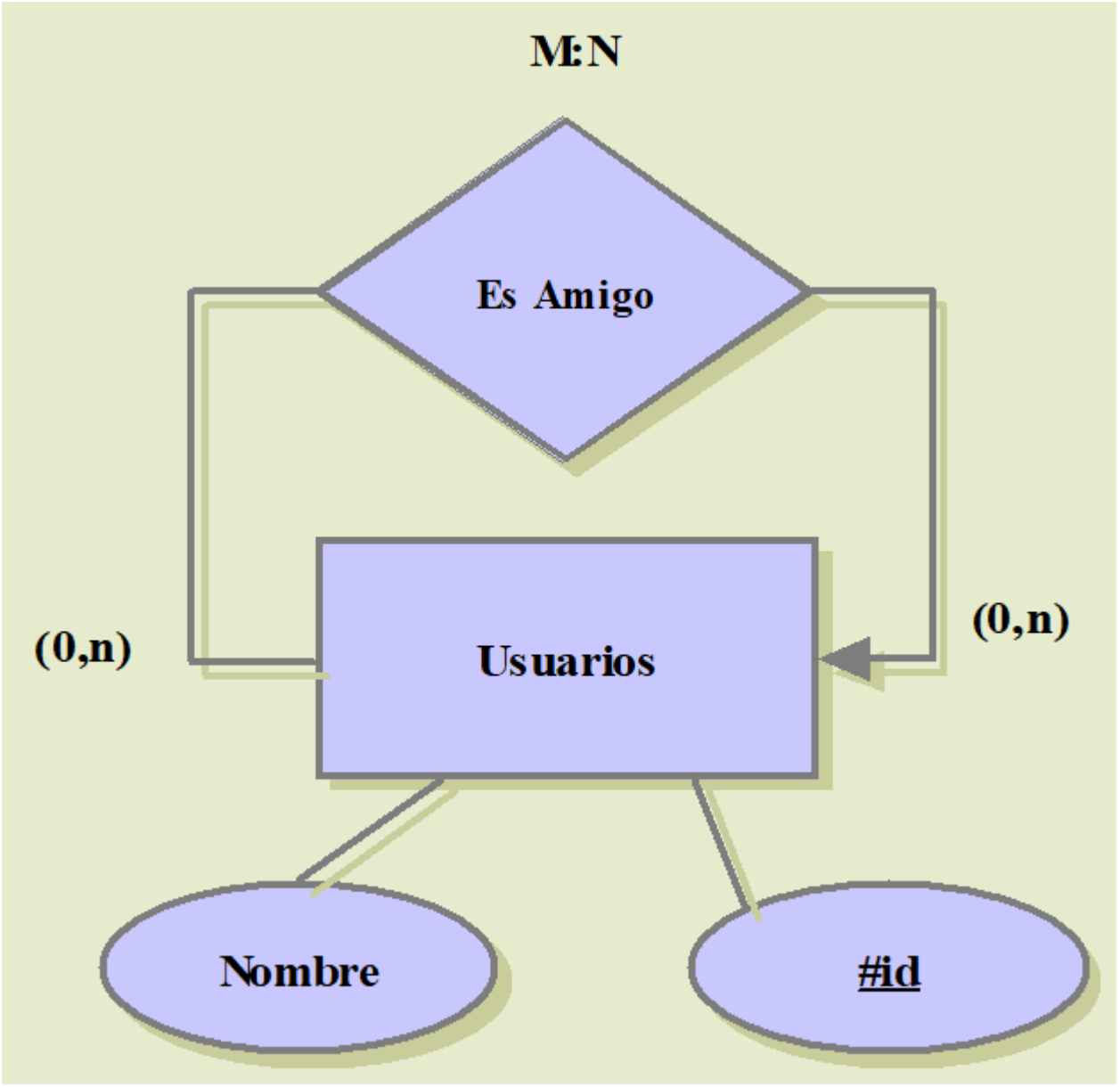


Es el caso más común: se propaga la clave de la tabla *Empleados* hacia sí misma, cambiándole el nombre, de modo que pasa a ser una clave ajena que referencia a su propia tabla. Si la participación en el lado «muchos» es opcional (el jefe máximo no tiene supervisor), la clave ajena admite `NULL`.

> Empleados(**#id**, Nombre, id_supervisor*)
>
> - `Empleados.id_supervisor` → `Empleados.id` (NULL, B:N, M:C)

4.8.3 Caso C. M:N

Figura 4-19.



La relación vincula ocurrencias de *Usuarios* con otras ocurrencias de *Usuarios*. La clave primaria de la nueva tabla está formada por la clave de *Usuarios* dos veces, cambiando el nombre de una de ellas:

> Usuarios(**#id**, Nombre)
>
> EsAmigo(**#id_usuario**, **#id_amigo**)
>
> - `EsAmigo.id_usuario` → `Usuarios.id` (NOT NULL, B:R, M:C)
> - `EsAmigo.id_amigo` → `Usuarios.id` (NOT NULL, B:R, M:C)

4.9 Transformación de las relaciones débiles

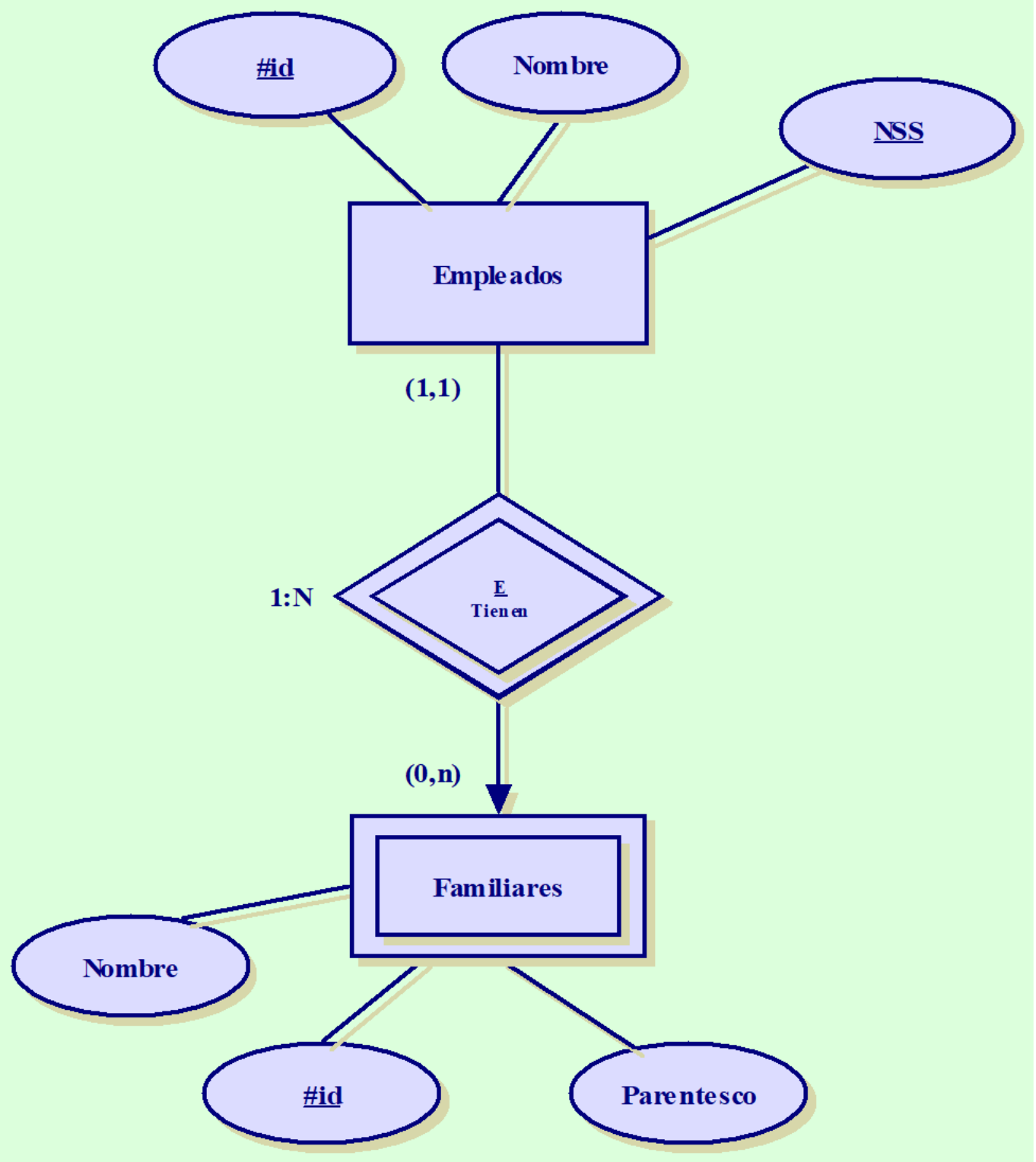
La transformación de las relaciones débiles (y de las entidades débiles) es parecida a lo visto hasta ahora, con algunas diferencias. Hay que tener en cuenta el tipo de dependencia:

- **Dependencia en existencia.** Se propaga la clave de la entidad fuerte a la débil como clave ajena, con borrado en cascada: si desaparece la entidad fuerte, desaparecen las ocurrencias débiles que dependen de ella.

- **Dependencia en identificación.** Se aplica la regla anterior y, además, la clave primaria de la entidad débil es la concatenación de su propia clave parcial (discriminador) y la clave primaria de la entidad fuerte.

4.9.1 Dependencia en existencia

Figura 4-20.

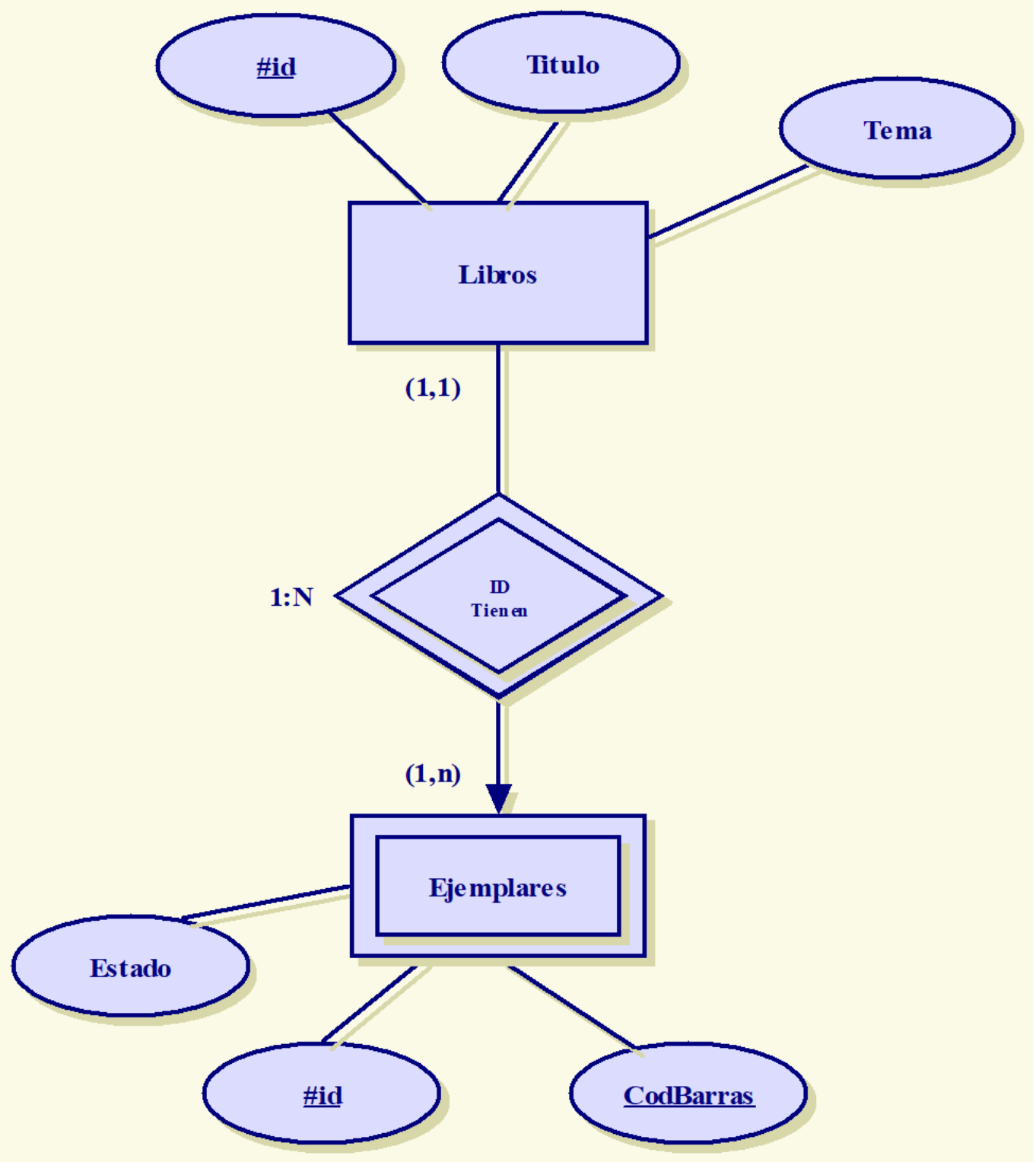


> Empleados(**#id**, Nombre, NSS)
>
> Familiar(**#id**, Nombre, Parentesco, id_empleado)

> - `Familiar.id_empleado` → `Empleados.id` (NOT NULL, B:C, M:C)

4.9.2 Dependencia en identificación

Figura 4-21.



> Libros(**#id**, Titulo, Tema)
>
> Ejemplares(**#num_ejemplar**, **#id_libro**, CodBarras, Estado)
>
> - `Ejemplares.id_libro` → `Libros.id` (NOT NULL, B:C, M:C)

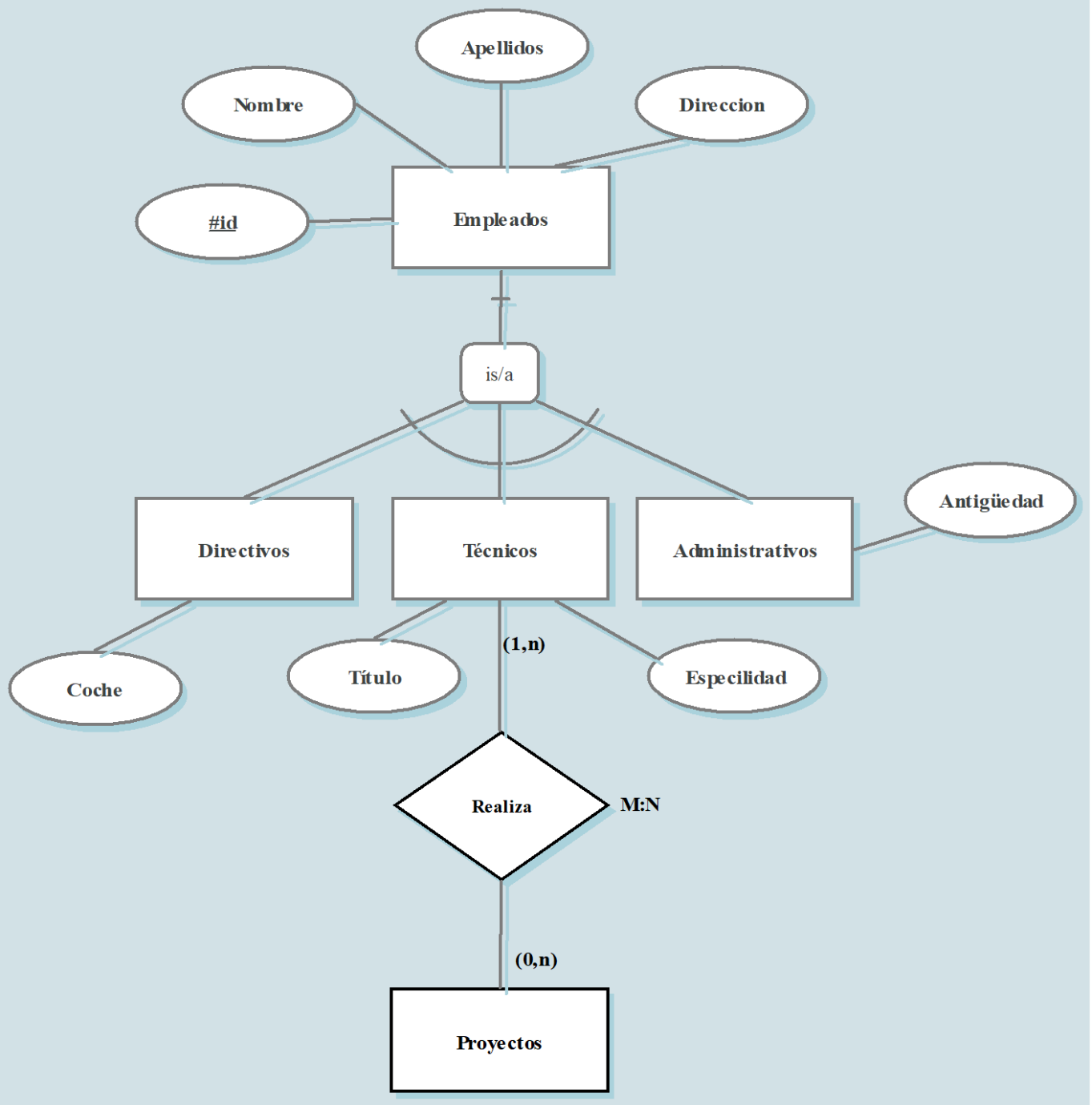
> En el original la clave parcial se llamaba `id`; se renombra a `num_ejemplar` para distinguirla claramente de la clave primaria de `Libros`. `num_ejemplar` solo es único dentro de cada libro.

4.10 Transformación de las jerarquías

Para las jerarquías (generalización/especialización) se aplica la siguiente regla:

- Cada entidad (supertipo y subtipos) se transforma en una tabla.
- La tabla del supertipo tiene como clave primaria la clave del supertipo y contiene los atributos comunes.
- Las tablas de los subtipos heredan la clave primaria del supertipo y contienen además sus atributos específicos.
- Las claves primarias heredadas por los subtipos son a su vez claves ajenas que referencian a la tabla del supertipo.

Ejemplo (figura 4-22).



> Empleados(**#id**, Nombre, Apellidos, Direccion)
>
> Directivo(**#id_empleado**, Coche)
>
> - `Directivo.id_empleado` → `Empleados.id` (NOT NULL, B:C, M:C)
>
> Tecnico(**#id_empleado**, Titulo, Especialidad)
>
> - `Tecnico.id_empleado` → `Empleados.id` (NOT NULL, B:C, M:C)
>
> Administrativo(**#id_empleado**, Antigüedad)
>
> - `Administrativo.id_empleado` → `Empleados.id` (NOT NULL, B:C, M:C)
>
> Realiza(**#id_proyecto**, **#id_empleado**)
>
> - `Realiza.id_proyecto` → `Proyectos.id` (NOT NULL, B:R, M:C)

> - `Realiza.id_empleado` → `Tecnico.id_empleado` (NOT NULL, B:R, M:C) *(solo referencia al subtipo Tecnico)*

Observación. La transformación en una tabla por entidad es la más general, pero no la única: según la jerarquía (total o parcial, exclusiva o solapada) puede ser preferible una única tabla con todos los atributos y un atributo discriminador, o solo tablas para los subtipos (ver 4.12).

4.11 Resumen de las reglas de transformación

Elemento del modelo E/R	Resultado en el modelo relacional
Entidad	Una tabla; los atributos pasan a columnas y el identificador principal a clave primaria.
Atributo compuesto	Solo se conservan los atributos que lo componen.
Atributo multivaluado	Nueva tabla (entidad débil) con la clave de la entidad y el atributo como clave primaria.
Relación M:N	Nueva tabla con las claves de ambas entidades (clave primaria compuesta).
Relación 1:N (mínima 1)	Se propaga la clave del lado 1 al lado N como clave ajena `NOT NULL`.
Relación 1:N (mínima 0)	Nueva tabla (o clave ajena con `NULL` como alternativa).
Relación 1:1 (1,1)-(1,1)	Se propaga la clave de cualquiera de las dos (clave ajena `UNIQUE`).
Relación 1:1 (0,1)-(0,1)	Nueva tabla con la clave de una como primaria y la de otra como ajena `UNIQUE`.
Relación 1:1 (0,1)-(1,1)	La clave de la entidad (0,1) se propaga a la tabla de la entidad (1,1) (`UNIQUE`, `NOT NULL`).
Relación ternaria / n-aria	Nueva tabla con las claves de todas las entidades; la clave primaria excluye una de las entidades con «uno».
Relación reflexiva	Clave ajena a la propia tabla (1:N) o nueva tabla con la clave dos veces (M:N, 1:1 con (0,1) y (0,1)).
Entidad débil	Clave de la fuerte como clave ajena con borrado en cascada; en identificación forma parte de la clave primaria.
Jerarquía	Una tabla por supertipo y subtipo; la clave del subtipo es primaria y ajena al supertipo.

4.12 Ampliaciones

> Esta sección es una **ampliación propuesta** sobre el documento original. Conviene revisarla antes de incorporarla al temario.

4.12.1 Elección de las políticas de borrado y actualización

Situación	Política recomendada
Entidad débil (existencia o identificación), atributo multivaluado, subtipo de una jerarquía	`B:C`, `M:C`
Tabla de relación M:N (referencias a entidades fuertes de negocio)	`B:R`, `M:C` (se evita perder el histórico sin querer)
Clave ajena de participación obligatoria (1,1)	`B:R`, `M:C`
Clave ajena de participación opcional (0,1) con la tabla sin clave en la PK	`B:N`, `M:C`
Datos históricos o auditables	`B:R` (o borrado lógico con un campo `activo`/`fecha_baja`)

Como las claves primarias suelen ser inmutables (claves sustitutas, `AUTO_INCREMENT`), `M:C` casi nunca llega a ejecutarse; es la opción más segura cuando se modifica.

4.12.2 Ejemplo completo de paso a SQL

A partir del caso *Usuario – Escucha – Cancion* (4.3):

```sql
CREATE TABLE Cancion (
  id     INT AUTO_INCREMENT PRIMARY KEY,
  Titulo VARCHAR(100) NOT NULL
);

CREATE TABLE Usuario (
  id     INT AUTO_INCREMENT PRIMARY KEY,
  Nombre VARCHAR(60) NOT NULL
);

CREATE TABLE Escucha (
  id_cancion INT NOT NULL,
  id_usuario INT NOT NULL,
  Fecha      DATE NOT NULL,
  Hora       TIME NOT NULL,
  PRIMARY KEY (id_cancion, id_usuario, Fecha, Hora),
  CONSTRAINT fk_escucha_cancion FOREIGN KEY (id_cancion)
    REFERENCES Cancion (id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_escucha_usuario FOREIGN KEY (id_usuario)
    REFERENCES Usuario (id) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

(Aquí se ha aplicado la observación de 4.3: `Fecha` y `Hora` forman parte de la clave para permitir escuchas repetidas.)

4.12.3 Alternativas de transformación de jerarquías

Alternativa	Cuándo es adecuada	Inconveniente
Una tabla por entidad (la del tema)	Caso general; jerarquía parcial o solapada	Requiere `JOIN` para reconstruir un subtipo
Una sola tabla con atributo discriminador	Subtipos con pocos atributos específicos	Muchos valores `NULL`
Solo tablas para los subtipos	Jerarquía total y exclusiva	Se duplican los atributos comunes y es difícil referenciar al supertipo

4.12.4 Fusión de tablas en relaciones 1:1 (1,1)-(1,1)

Cuando ambas participaciones son obligatorias en las dos entidades, a menudo es más coherente **fusionar ambas entidades en una sola tabla**, sobre todo si no existe motivo para separarlas. Evita el problema de la inserción circular de claves ajenas.

4.12.5 Restricciones que no se plasman en el modelo (RA6.h)

El diseño lógico no siempre recoge todas las reglas del enunciado. Hay que **documentarlas** para implementarlas posteriormente con `CHECK`, triggers o la aplicación:

- Mínimos y máximos de cardinalidad mayores que 1 (por ejemplo, «un equipo tiene al menos 11 jugadores»).
- Exclusividad o totalidad de jerarquías.
- Simetría y no reflexividad (por ejemplo, `EsAmigo`, `EsEsposo`).
- Relaciones de agregación: la relación entre una relación y una entidad se transforma tratando la relación agregada como una entidad cuya clave es la de su tabla.

4.12.6 Esquema de comprobación para el alumno

Tras transformar un diagrama E/R, comprobar:

1. Cada entidad tiene su tabla con clave primaria.
2. Todos los atributos son atómicos (compuestos y multivaluados resueltos).
3. Cada relación se ha transformado según su tipo y cardinalidades mínimas.
4. Toda clave ajena referencia una clave primaria (o `UNIQUE`) existente, del mismo tipo.
5. Cada clave ajena tiene indicadas las políticas de nulos, borrado y actualización, y son coherentes entre sí.
6. Se han documentado las restricciones que el modelo no puede expresar.

4.13 Referencias normativas

- **Real Decreto 405/2023, de 29 de mayo**, por el que se actualizan los títulos de la formación profesional del sistema educativo de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma y Técnico Superior en Desarrollo de Aplicaciones Web, de la familia profesional Informática y Comunicaciones, y se fijan sus enseñanzas mínimas («BOE» núm. 132, de 3 de junio de 2023; BOE-A-2023-13221; ELI: https://www.boe.es/eli/es/rd/2023/05/29/405). Sustituye el articulado y los anexos del

Real Decreto 686/2010, de 20 de mayo, que estableció el título de Técnico Superior en Desarrollo de Aplicaciones Web, y redacta el módulo profesional **0484. Bases de datos** (12 ECTS, 105 horas).

- **Resultado de aprendizaje 6** del módulo 0484: «Diseña modelos relacionales normalizados interpretando diagramas entidad/relación», con los criterios de evaluación a) a h) (a: herramientas gráficas; b: tablas; c: campos; d: relaciones; e: campos clave; f: reglas de integridad; g: normalización; h: restricciones no plasmables en el diseño lógico).
- **Contenidos básicos** asociados: «Interpretación de Diagramas Entidad/Relación» (modelo E/R, cardinalidades, debilidad, generalización y especialización, agregación, paso del diagrama E/R al modelo relacional, restricciones semánticas y normalización) y «Bases de datos relacionales» (claves primarias, valor `NULL`, claves ajenas).

Notas de verificación

- Las referencias anteriores se han contrastado con el texto del BOE (RD 405/2023) y con `normativa/contenidos_oficiales_modulo_bases_de_datos.md`.
- El documento original afirmaba que el RD 405/2023 «redacta el módulo 0484»; es correcto, pero en realidad lo hace sustituyendo el articulado y los anexos del RD 686/2010 (cuyo título se mantiene).
- **No verificable aquí:** el currículo autonómico (Andalucía) que concreta el módulo en el centro. Debe consultarse en el BOJA la orden vigente del ciclo DAW. No se cita ninguna orden concreta para no inventar referencias.
- Los temas de referencia `ia_dwes_2627\temas\tema01_introduccion_a_los_sistemas_de_informacion.md` y `tema02_modelado_de_datos.md` **no existen** con esos nombres (la carpeta contiene temas del módulo DWES 0613, de otro módulo). Se ha seguido su criterio general (citar RD 686/2010 y RD 405/2023, advertir de que el currículo autonómico debe contrastarse) pero no se ha podido cotejar su contenido.